

el trabajo de la implementación precedente. Por ejemplo, cuando el ciclo N se desvía de su diseño (lo cual ocurrirá inevitablemente), el diseño final basado en la implementación puede utilizarse en los modelos de análisis y diseño del ciclo N+1.

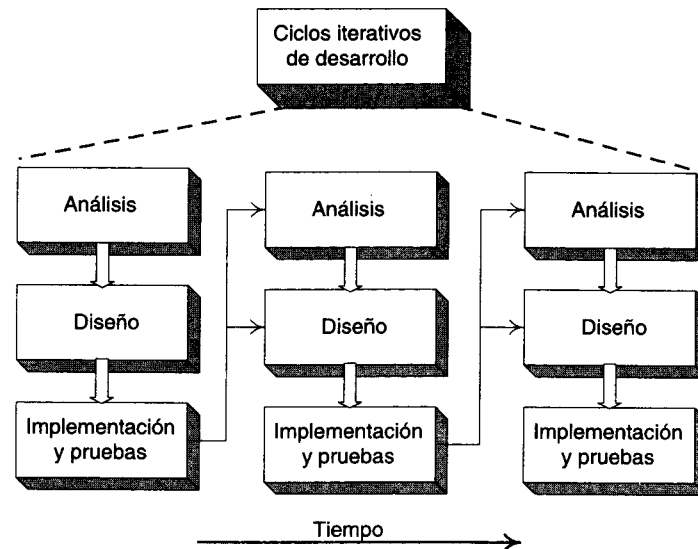


Figura 23.1 La implementación en un ciclo de desarrollo influye en el diseño ulterior.

Por ello, una actividad inicial de un ciclo de desarrollo consiste en sincronizar los artefactos; los diagramas del ciclo N no coincidirán con el código final de este ciclo, de modo que han de ser sincronizados antes de extenderlos mediante otros resultados del análisis y del diseño.

23.2.3 Cambios de código, las herramientas CASE y la ingeniería inversa

Es conveniente que los diagramas generados en la fase de diseño sean actualizados semi-automáticamente para que incluyan los cambios en la siguiente fase de codificación. En teoría, esto debería hacerse con una herramienta de CASE (*Computer-aided software engineering*, ingeniería de software asistida por computadora), la cual puede leer el código fuente (Java, entre otros) y producir automáticamente —por ejemplo— los diagramas de clases y de colaboración. Éste es un aspecto de la **ingeniería inversa**, o sea la actividad consistente en generar modelos lógicos partiendo de un código fuente ejecutable.

23.3 Mapeo de diseños para codificación

Para implementar un lenguaje de programación orientado a objetos se requiere escribir un código fuente para:

- definiciones de clase
- definiciones de métodos

En las siguientes secciones explicaremos su creación en Java (un caso muy representativo); en el capítulo 24 veremos casos concretos de una implementación completa.

23.4 Creación de las definiciones de clase a partir de los diagramas de clases del diseño

Los diagramas de clases del diseño describen por lo menos el nombre de las clases, las superclases, las etiquetas de los métodos y los atributos simples de una clase. Esa información es suficiente para formular una definición básica de clase en un lenguaje orientado a objetos. Más adelante hablaremos de la incorporación de información sobre la interfaz y sobre el espacio destinado al nombre, entre otros detalles.

23.4.1 Definición de una clase con métodos y con atributos simples

Como se aprecia en la figura 23.2, un diagrama de clases del diseño facilita mapear las definiciones básicas de los atributos (simples variables de instancias en Java) y las etiquetas de métodos en la definición de *VentasLineadeProducto* de Java.

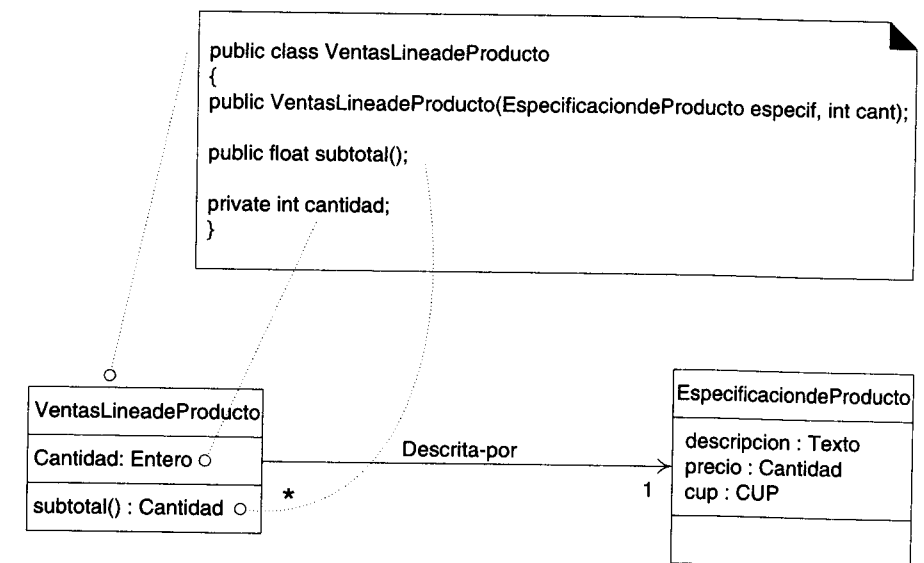


Figura 23.2 VentasLineadeProducto en Java.

Nótese que el constructor de Java *VentasLineadeProducto(...)* ha sido agregado. Deriva del hecho de que se le envía un mensaje *crear(especific, cant)* a *VentasLineadeProducto*

en el diagrama de colaboración *introducirProducto*. En este lenguaje ello indica que se requiere un constructor que soporte los parámetros. El método *crear* a menudo se excluye en el diagrama de clase por ser muy común y por sus múltiples interpretaciones, según el lenguaje objetivo.

Observe también que transformamos de *Cantidad* a simple punto *flotante* el tipo de retorno del método *subtotal*. Suponga que, en el trabajo inicial de codificación, el diseñador no quiere dedicarle tiempo a la clase *Cantidad* y que pospondrá esta actividad.

23.4.2 Adición de los atributos de referencia

El **atributo de referencia** es aquel que remite a otro objeto complejo, no a un tipo primitivo como String, Number, etcétera.

Los atributos de referencia de una clase están indicados por las asociaciones y la navegabilidad en un diagrama de clases.

Por ejemplo, una clase *VentasLineadeProducto* tiene una asociación a *EspecificaciondeProducto*, con navegabilidad respecto a esta última. Se acostumbra interpretarla como atributo de referencia en la clase *VentasLineadeProducto* que remite a la instancia *EspecificaciondeProducto* (figura 23.3).

En Java, lo anterior significa que denota una variable de instancia que se refiere a la instancia *EspecificaciondeProducto*.

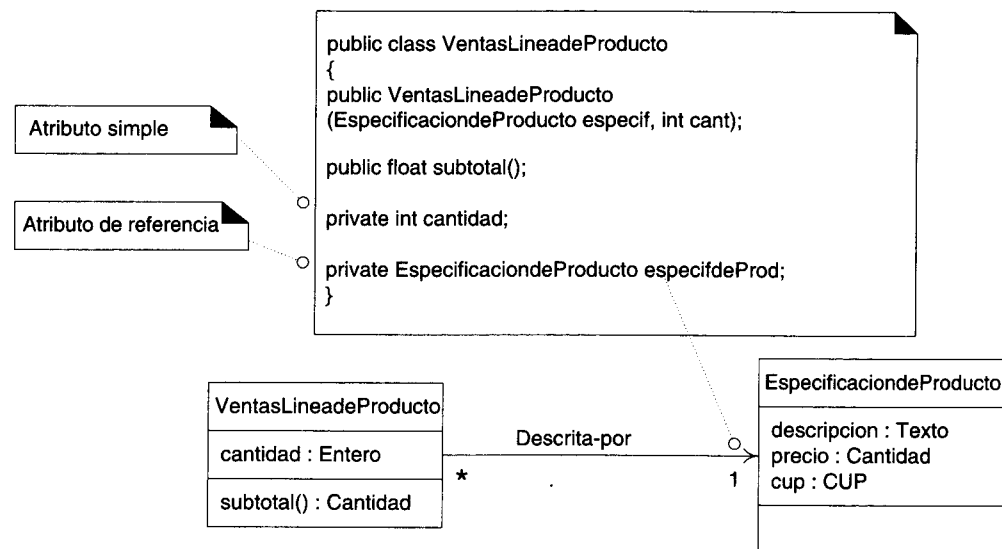


Figura 23.3 Agregación de atributos de referencia.

Nótese que a menudo los atributos de referencia de una clase están implícitos, no explícitos, en un diagrama de clases orientado al diseño.

Por ejemplo, aunque hemos incorporado una variable de instancia a la definición de *VentasLineadeProducto* en Java para apuntar a *EspecificaciondeProducto*, no se declara explícitamente como atributo en la sección dedicada a ellos dentro de la casilla que representa la clave. Hay una visibilidad *sugerida* de atributos —indicada por la asociación y la navegabilidad— que se define explícitamente como un atributo durante la fase de generación del código.

23.4.3 Atributos de referencia y nombres de los papeles

La siguiente iteración analizará el concepto de los nombres de papeles en los diagramas de estructura estática. A cada fin o extremo de una asociación se le da el nombre de papel. En pocas palabras, el **nombre de papel** es aquel que identifica el papel y muchas veces contiene algún contexto semántico respecto a la naturaleza del papel.

Si el nombre de un papel se encuentra en un diagrama de clase, utilícelo como criterio para elegir el nombre del atributo de referencia durante la generación del código, según se señala en la figura 23.4.

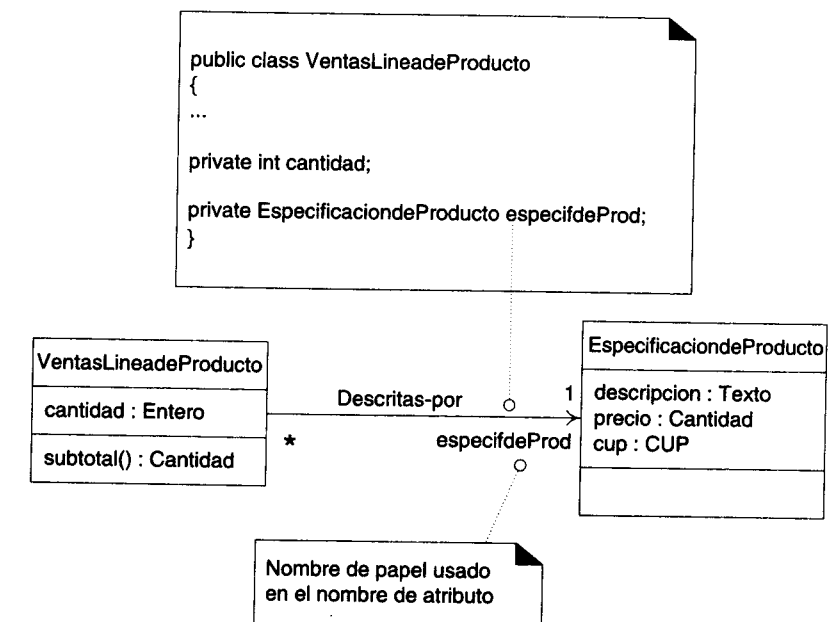


Figura 23.4 Con los nombres de papeles pueden generarse nombres de variables de instancias.

23.5 Creación de métodos a partir de los diagramas de colaboración

Un diagrama de colaboración muestra los mensajes que se envían en respuesta a una llamada al método. La secuencia de los mensajes se traduce a una serie de enunciados en la definición del método.

El diagrama de colaboración *introducirProducto* de la figura 23.5 nos servirá para dar un ejemplo concreto de la definición del método *introducirProducto* en Java.

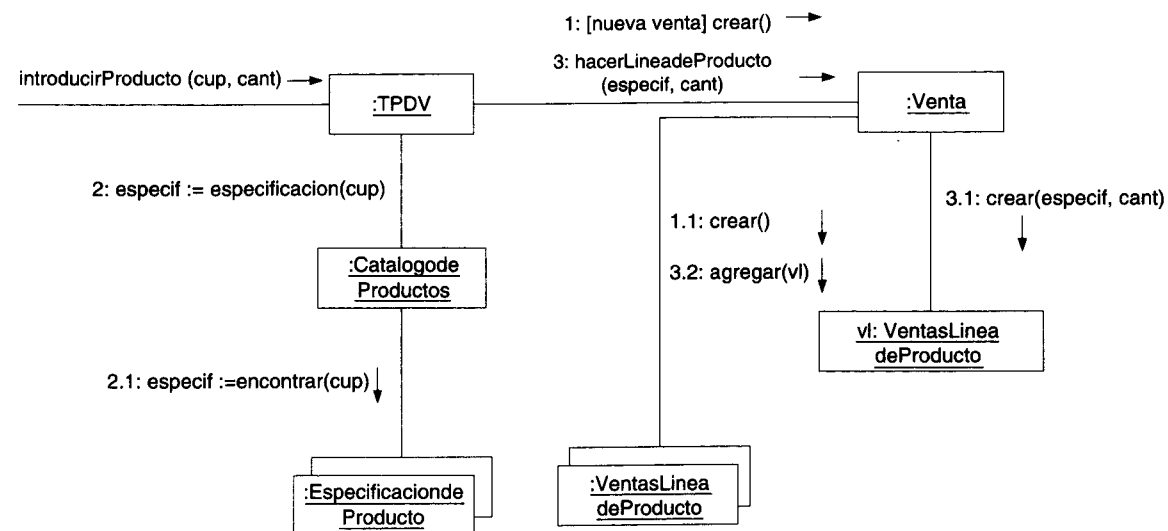


Figura 23.5 El diagrama de colaboración introducirProducto.

En este ejemplo utilizaremos la clase *TPDV*. En la figura 23.6 se incluye una definición de Java.

23.5.1 El método TPDV--introducirProducto

El mensaje *introducirProducto* se envía a una instancia *TPDV* y, por tanto, el método *introducirProducto* se define en la clase *TPDV*.

```
public void introducirProducto(int cup, int cant)
```

Mensaje 1: conforme al diagrama de colaboración, en respuesta al mensaje *introducirProducto* el primer enunciado tiene por objeto crear condicionalmente una nueva *Venta*.¹

```
if ( esNuevaVenta() ) { Venta = new Venta(); }
```

¹ Por razones de brevedad, se prescinde de los métodos de acceso aunque generalmente se recomienda utilizarlos.

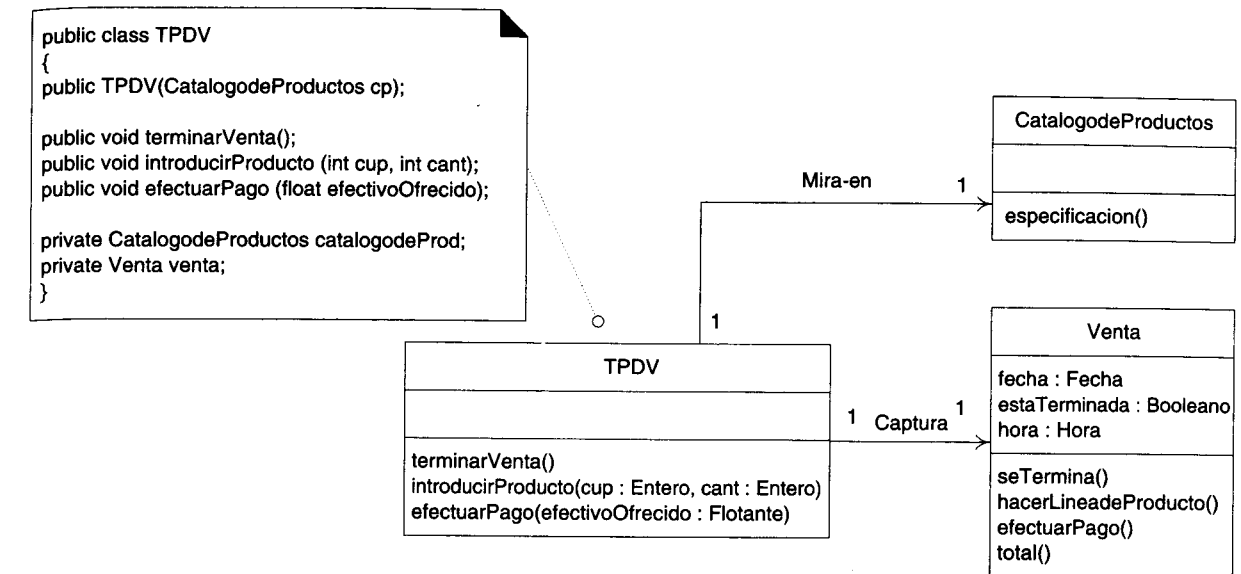


Figura 23.6 La clase TPDV.

Mensaje 2: Segundo, enviamos un mensaje *especificación* a *CatalogodeProductos* para recuperar una instancia *EspecificaciondeProducto*.

```
EspecificaciondeProducto especif =  
catalogodeProd.especificacion(cup);
```

Mensaje 3: Tercero, enviamos a la *Venta* el mensaje *hacerLineadeProducto*.

```
venta.hacerLineadeProducto (especif, cant);
```

En resumen, en el método de Java los mensajes en secuencia dentro de un método se mapean en una proposición como se indica en el diagrama de colaboración.

El método íntegro *introducirProducto* y su relación con el diagrama de colaboración se muestran en la figura 23.7.

23.5.2 El método TPDV--esNuevaVenta

La definición del método *TPDV--introducirProducto* dio origen a un nuevo método en la clase *TPDV*: *esNuevaVenta*. Éste es un ejemplo pequeño de cómo, durante la fase de codificación, aparecen cambios en el diseño. Posiblemente este método hubiera sido descubierto durante la fase inicial de la solución, pero lo importante es que inevitablemente se producirán cambios mientras se efectúa la programación.

En un primer intento, este método de ayuda aplicará una prueba basándose en que la variable de la instancia *ventas* sea o no *nula*.

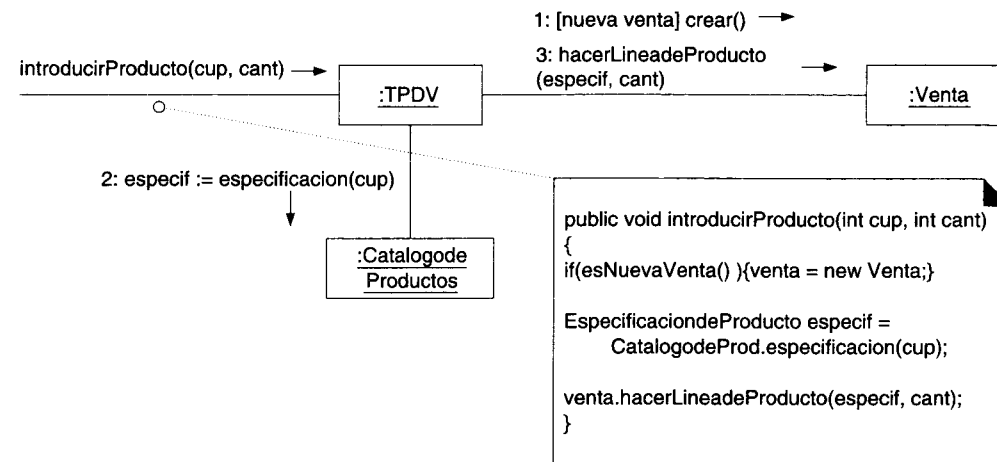


Figura 23.7 El método introducirProducto.

Por tanto,

```
private boolean esNuevaVenta()
{
    return (venta == null);
}
```

¿Por qué no simplemente codificamos esta prueba y la incorporamos al método *introducirProducto*? No lo hacemos porque se basa en una decisión de diseño sobre la representación de la información. En términos generales, lo más conveniente es incluir en métodos de ayudante las expresiones que dependen de este tipo de decisiones; se hace eso para reducir al mínimo el cambio en caso de que se modifique la representación. Además, para un lector del código la prueba *esNuevaVenta* resulta más informativa en lo tocante al propósito semántico. Considere la relativa claridad semántica de las siguientes expresiones alternas:

```
if ( esNuevaVenta() )...

// a diferencia de

if (venta == null)...
```

El lector al reflexionar se dará cuenta de que esta prueba no es adecuada en el caso general. Por ejemplo, ¿qué sucedería si se terminó una venta y está a punto de empezar otra? De ser así, el atributo *venta* no será *nulo*; apuntará a la última venta. En consecuencia, se necesita alguna prueba adicional para determinar si se trata de una nueva venta. Para resolver este problema supongamos que, si la venta actual está en el estado *terminada*, podrá empezar otra venta. Puede efectuarse fácilmente un cambio, si en una iteración posterior esta consideración resulta ser una regla inapropiada de negocios.

Por tanto,

```
private boolean esNuevaVenta()
{
    return ( venta == null ) || sale.isComplete();
}
```

Este cambio extemporáneo del significado de “una nueva venta” nos permite apreciar una vez más la ventaja de encapsular su prueba dentro de un método de ayudante: el método no requiere más que un solo cambio.

23.6 Actualizaciones de las definiciones de clases

A partir de estas decisiones de codificaciones, habrá que agregar un nuevo método a la definición de la clase *TPDV*: *esNuevaVenta*. Deberíamos actualizar el diagrama de clases del diseño que describe la clase *TPDV* para que refleje este cambio de código. Si estamos utilizando una herramienta de CASE con las capacidades de la ingeniería inversa, tal vez no sea difícil regenerar los diagramas basándonos para ello en el estado del código. En caso contrario se requerirá el mantenimiento manual. Aplicar esta técnica a los diagramas es de gran utilidad; pero como rara vez se lleva a cabo de manera adecuada y resulta muy costoso, se justifica entonces invertir en una herramienta de CASE capaz de aplicar esta tecnología para generar los diagramas de UML.

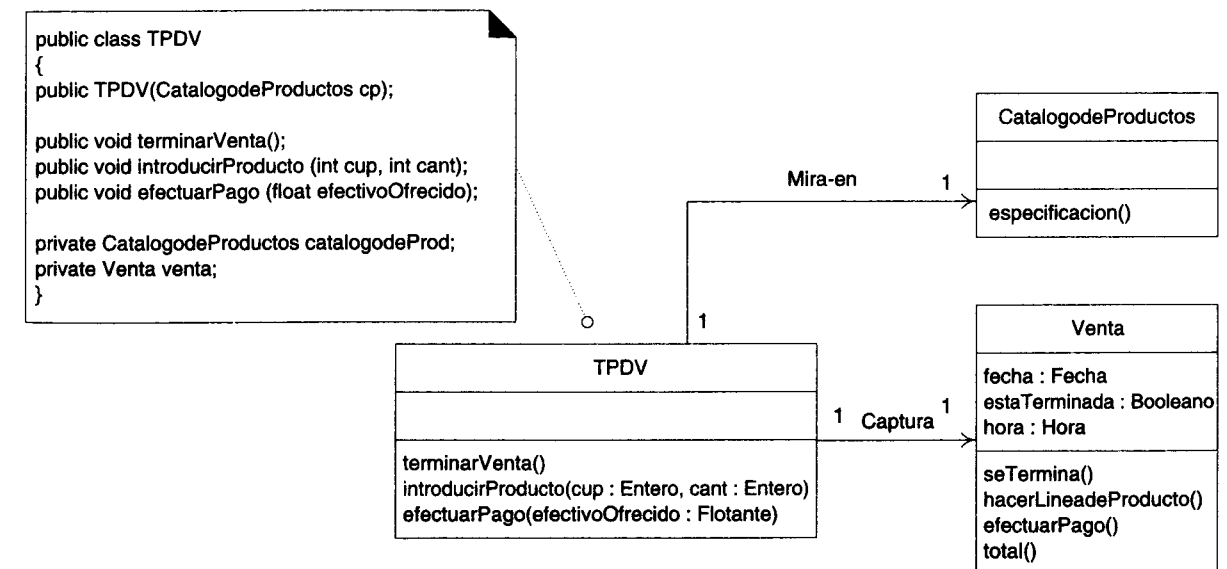


Figura 23.8 Definición actualizada de la clase TPDV.

23.7 Las clases de contenedor/colección en código

A menudo un objeto debe conservar la visibilidad de un grupo de otros objetos; la necesidad de ello se pone en evidencia generalmente con el valor de multiplicidad que presenta un diagrama de clase: puede ser mayor que uno. Por ejemplo, una *Venta* ha de conservar la visibilidad de un grupo de instancias *VentasLineadeProducto*, como se observa en la figura 23.9.

En los lenguajes de programación orientados a objetos estas relaciones a menudo se implementan introduciendo un contenedor o colección intermedios. La clase del lado "uno" define un atributo de referencia que apunta a una instancia contenedor/colección, el cual contiene instancias de la clase del lado "muchos".

Por ejemplo, el JDK de Java incluye clases de contenedor como (*Hashtable*) y *Vector*. Por medio de *Vector* la clase *Venta* puede definir un atributo que conserva una lista ordenada de las instancias *VentasLineadeProducto*.

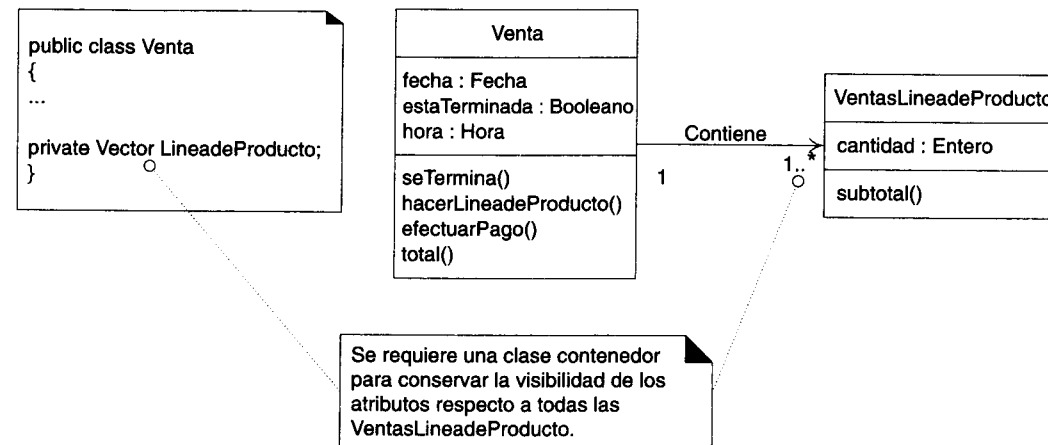


Figura 23.9 Agregación de un contenedor.

Los requerimientos influyen al elegir una clase de contenedor; la consulta basada en claves exige emplear una *Hashtable*, una lista ordenada creciente requiere un *Vector* y así sucesivamente.

23.8 Manejo de las excepciones y de los errores

Hasta ahora no hemos hablado del manejo de errores en la obtención de una solución. Lo hicimos intencionalmente con el propósito de concentrarnos en las cuestiones fundamentales de asignar responsabilidades y en el diseño orientado a objetos. Pero en una aplicación concreta conviene ocuparse del manejo de errores en la fase de diseño.

Por ejemplo, podemos comentar los contratos anexándoles una breve explicación de las situaciones ordinarias de errores y el plan general de solución.

El lenguaje UML no cuenta con una notación especial para indicar las excepciones. Más bien, se indican con la notación de los diagramas de colaboración basados en los mensajes. Un diagrama de colaboración puede comenzar con un mensaje que representa una excepción ocurrida.

23.9 Definición del método Venta--hacerLineadeProducto

Un último ejemplo: el método *hacerLineadeProducto* también puede escribirse observando atentamente el diagrama de colaboración *introducirProducto*. En la figura 23.10 se incluye una versión abreviada de este diagrama, con el método correspondiente de Java.

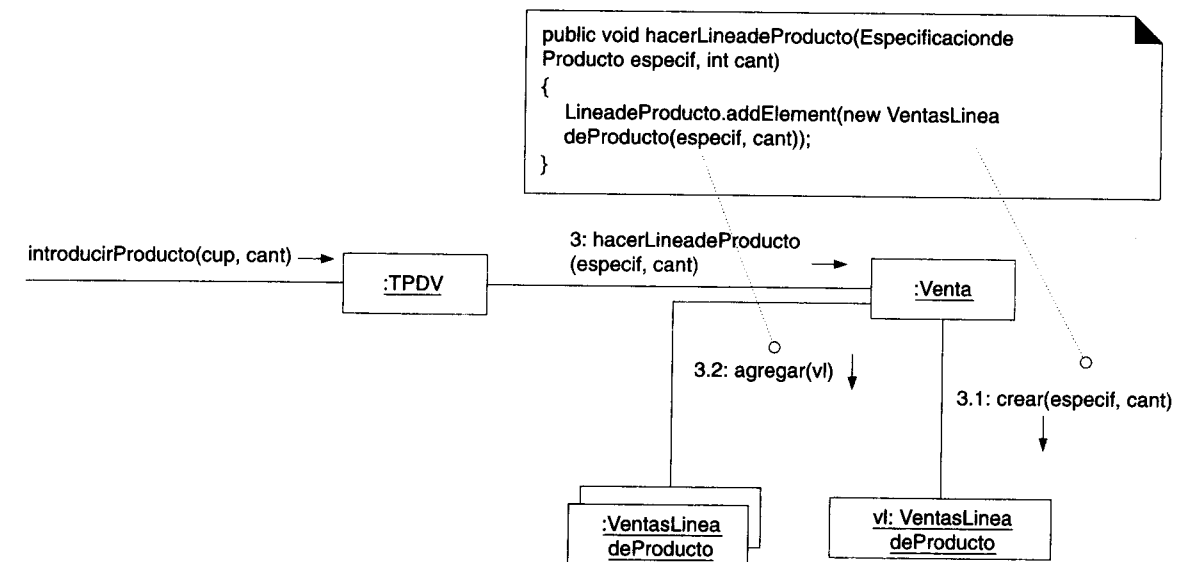


Figura 23.10 Método Venta--hacerLineadeProducto.

Este método presenta otro ejemplo de modificación del código respecto al diagrama de colaboración: el mensaje genérico *agregar* ha sido traducido al mensaje específico de Java *addElement*. En este tipo de diagramas también podemos emplear los mensajes propios del lenguaje o de la biblioteca, como *addElement*.

23.10 Orden de la implementación

Es necesario implementar las clases (y, en teoría, probarlas totalmente de manera individual), de la menos acoplada a la más acoplada (figura 23.11). Por ejemplo, las posi-

bles primeras clases para implementar son *Pago* o *EspecificaciondeProducto*. Vienen después las clases que dependen únicamente de las implementaciones anteriores: *CatalogodeProductos* o *VentasLineadeProducto*.

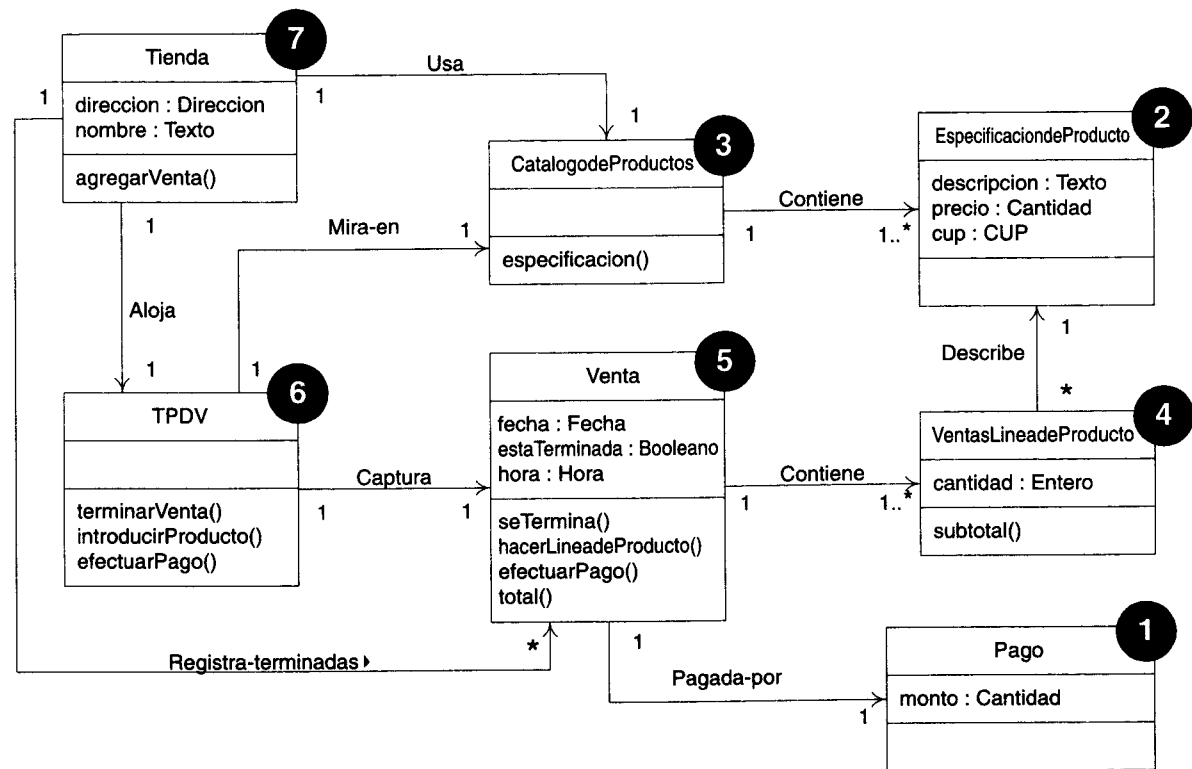


Figura 23.11 Posible orden de implementación de clases.

23.11 Resumen del mapeo del diseño a la codificación

Es muy sencillo el proceso de traducir a definiciones de clases los diagramas del diseño y también el proceso de traducir a métodos los diagramas de colaboración. En la fase de programación todavía pueden tomarse muchas decisiones, efectuarse muchos cambios de diseño y realizarse una exploración muy extensa; pero, en teoría, la arquitectura global y las decisiones más importantes ya se establecieron totalmente antes que comience la fase de codificación.

SOLUCIÓN EN PROGRAMA DE JAVA

24.1 Introducción a la solución convertida en programa

En este capítulo incluimos ejemplos de la solución convertida en programa de la capa de objetos del dominio para el primer ciclo de desarrollo de la aplicación punto de venta. La generación de código proviene principalmente de los diagramas de clases del diseño y de los diagramas de colaboración que se definieron en la fase de diseño; la generación de código se basa en los principios del mapeo de diseño a codificación que expusimos en el capítulo anterior.

El punto principal de este listado es el siguiente: la traducción relativamente directa de los artefactos del diseño a la creación de código. El código define un caso sencillo de prueba: no pretende ejemplificar un programa robusto y totalmente desarrollado de Java con sincronización, manejo de excepciones u otras características.

Clase Pago

```
package tpdv;

public class Pago
{
    private float monto;
    public Pago( float efectivoOfrecido )
    {
        this.monto = efectivoOfrecido;
    }
    public float obtenerMonto() { return monto; }
}
```

Clase CatalogodeProductos

```
package tpdv;
```

```

import java.util.*;

public class CatalogodeProductos
{
    private Hashtable EspecificacionesdeProducto
        = new Hashtable();

    public CatalogodeProductos()
    {
        EspecificacionesdeProducto ep =
            New EspecificaciondeProducto( 100, 1, "producto 1" );
        EspecificacionesdeProducto.put(new Integer( 100 ), ep );
        Ep = new EspecificaciondeProducto(200, 1, "producto 2");
        EspecificacionesdeProducto.put(new Integer( 200 ), ep );
    }

    public EspecificaciondeProducto obtenerEspecificacion
        ( int cup )
    {
        return (EspecificaciondeProducto)
            EspecificacionesdeProducto.get( new Integer ( cup ) );
    }
}

```

Clase TPDV

```

package tpdv;
import java.util.*;

class TPDV
{
    private CatalogodeProdutos catalogodeProductos;
    private Venta venta;

    public TPDV( CatalogodeProductos catalogo )
    {
        catalogodeProductos = catalogo;
    }

    public void terminarVenta()
    {
        venta.seTermina();
    }

    public void introducirProducto( int cup, int cantidad )
    {
        if( esNuevaVenta() )
        {
            venta = new Venta();
        }
        EspecificaciondeProducto especific =

```

```

        catalogodeProductos.especificacion( cup );
        venta.hacerLineadeProducto( especific, cantidad );
    }

    public void efectuarPago( float efectivoOfrecido )
    {
        venta.efectuarPago( efectivoOfrecido );
    }

    private boolean esNuevaVenta()
    {
        return ( venta == null ) || ( venta.estaTerminada() );
    }
}

```

Clase EspecificaciondeProducto

```

Package tpdv;

public class EspecificaciondeProducto
{
    private int    cup        = 0;
    private float  precio     = 0;
    private String descripcion = "";

    public EspecificaciondeProducto
        (int cup, float precio, String descripcion)
    {
        this.cup        = cup;
        this.precio     = precio;
        this.descripcion = descripcion;
    }

    public int obtenerCUP() { return cup; }

    public float obtenerPrecio() { return precio; }

    public String obtenerDescripcion() { return descripcion; }
}

```

Clase Venta

```

package tpdv;
import java.util.*;

class Venta
{
    private Vector LineadeProducto = new Vector();
    private Date fecha = new Date();
    private boolean estaTerminada = false;

```

```

private Pago pago;

public float obtenerSaldo()
{
    return pago.obtenerMonto() - total();
}

public void seTermina() { estaTerminada = true; }

public boolean estaTerminada() { return estaTerminada; }

public void hacerLineadeProducto
(EspecificaciondeProducto especific, int cantidad)
{
    LineadeProducto.addElement(
        new VentaLineadeProducto( especific, cantidad ) );
}

public float total()
{
    float total = 0;
    enumeration e = LineadeProducto.elements();
    while( e.hasMoreElements() )
    {
        total += ( (ventasLineadeProducto)
            e.nextElement() ).subtotal();
    }
    return total;
}

public void efectuarPago( float efectivoOfrecido )
{
    pago = new Pago( efectivoOfrecido );
}
}

```

Clase VentasLineadeProducto

```

package tpdv;

class VentasLineadeProducto
{
    private int cantidad;
    private EspecificaciondeProducto especifdeProd;

    public VentasLineadeProducto
        (EspecificaciondeProducto especific, int cantidad)
    {
        this.especifdeProd = especific;
        this.cantidad = cantidad;
    }
}

```

```

public float subtotal()
{
    return cantidad * especifdeProd.obtenerPrecio();
}
}

```

Clase Tienda

```

package tpdv;

class Tienda
{
    private CatalogodeProductos catalogodeProductos
        = new CatalogodeProductos();
    private TPDV tpdv = new TPDV ( CatalogodeProductos );

    public TPDV obtenerTPDV() { return tpdv; }
}

```


PARTE VI . FASE
DE ANÁLISIS (2)

ELECCIÓN DE LOS REQUERIMIENTOS DEL CICLO DE DESARROLLO 2

25.1 Requerimientos del ciclo de desarrollo 2

En el segundo ciclo de desarrollo de la aplicación del punto de venta revisamos los casos de uso *Comprar Productos* e *Iniciar*, pero les agregaremos una mayor funcionalidad para que se aproximen al caso de uso final y completo. Se acostumbra utilizar muchas veces un mismo caso de uso en varios ciclos de desarrollo y ampliarlo después para alcanzar la funcionalidad que se requiere.

El primer ciclo llevó a cabo muchas simplificaciones y por eso el problema no resultó excesivamente complejo. Una vez más —y por la misma razón—, consideraremos un grado relativamente pequeño de funcionalidad complementaria.

La funcionalidad complementaria consiste en que se dará soporte a los pagos en efectivo, con tarjeta de crédito y con cheque.

En un proyecto de desarrollo esto no será una decisión unánime de mayor funcionalidad en el segundo ciclo: otra posibilidad es actualizar de manera automática el inventario o un caso de uso totalmente diferente. Pero esta opción ofrece multitud de oportunidades valiosas de aprendizaje.

25.2 Suposiciones y simplificaciones

Las siguientes suposiciones y simplificaciones se adoptan en los dos casos de uso que hemos venido examinando.

**Comprar
Productos**

No se actualiza el inventario.

Es una tienda independiente, no parte de una organización más grande.

Captura manual de los códigos universales de producto (CUP); sin lector de código de barras.

Sin cálculos de impuestos.

Sin cupones.

Sin políticas especiales de precios.

El cajero no tiene que darse de alta en una cuenta.

No se llevan registros de cada cliente ni de sus hábitos de compra.

No se lleva un control de la gaveta de la caja.

En el recibo aparecen el nombre y la dirección de la tienda, la fecha y la hora de la venta.

En el recibo no aparecen la identificación del cajero ni la de TPDV.

Las ventas terminadas se anotan en el registro histórico o bitácora.

Sólo un pago, de un tipo, se utiliza en una venta.

Los pagos se realizan en su totalidad; no se aceptan pagos parciales ni pagos en abonos.

Se autorizan los pagos con cheque y con tarjeta de crédito.

Se utiliza un servicio diferente de autorización de crédito con cada tipo de crédito (Visa, MasterCard u otras tarjetas de crédito).

El mismo servicio de autorización de crédito se emplea con todos los cheques.

La terminal punto de venta se encarga de comunicarse con el servicio de autorización de crédito; el lector de las tarjetas de crédito es un dispositivo tonto que se limita a enviar a la terminal la información de la tarjeta.

La comunicación con un servicio externo se lleva a cabo a través de un módem. Debe marcarse un número telefónico cada vez que se emplee.

Generalmente un banco da los servicios de autorización de crédito.

Iniciar

Los pagos con cheque y con tarjeta de crédito deben cubrir el monto exacto de la venta.

Suponga que la fecha y la hora son las correctas.

Se requiere un mínimo de inicialización para brindar soporte al caso de uso *Comprar Productos*.

No se almacena información de inicialización en la base de datos.

CÓMO RELACIONAR CASOS MÚLTIPLES DE USO

Objetivos

- Relacionar los casos de uso con las asociaciones de aplicaciones y de extensiones.

26.1 Introducción

Podemos expresar los procesos relacionados con diversas formas de pago en la aplicación del punto de venta como casos independientes y relacionarlos con otros casos.

Su finalidad es preparar un diagrama actualizado que muestre estos casos de uso adicionales y sus relaciones.

El lenguaje UML tiene una notación especial para indicar las relaciones de los casos de uso; en este capítulo vamos a estudiarla y a describir un nuevo diagrama para el sistema.

26.2 Cuándo crear casos de uso independientes

En la explicación anterior sobre los casos de uso, los procesos de pago se anotaron en las subsecciones del caso *Comprar Productos*. También pudimos haberlos dividido en casos aislados. ¿Cuándo debería un paso importante o una actividad ramificada de un caso de uso escribirse como una parte de una subsección y no como un caso de uso aparte?

Escriba los pasos principales o las actividades ramificadas de un caso de uso como caso independiente cuando:

- Se duplican en otros casos.
- Son complejos y largos, y su separación facilita colocarlos en unidades manejables y comprensibles.

En la aplicación del punto de venta, los métodos de pago son las actividades ramificadas que pueden estar presentes también en otros casos como *Intercambiar Productos* o *Abonar a un Plan Apartado*. Por tanto, se justifica tratarlos como casos independientes y asignarles los siguientes nombres:

- *Pago en efectivo, Pago con tarjeta de crédito, Pago con cheque.*

26.3 Diagramas de casos de uso con las relaciones *usa*

Si un caso de uso inicia o contiene el comportamiento de otro, se dice que *usa* el segundo caso y que ambos se encuentran en una relación *usa*.

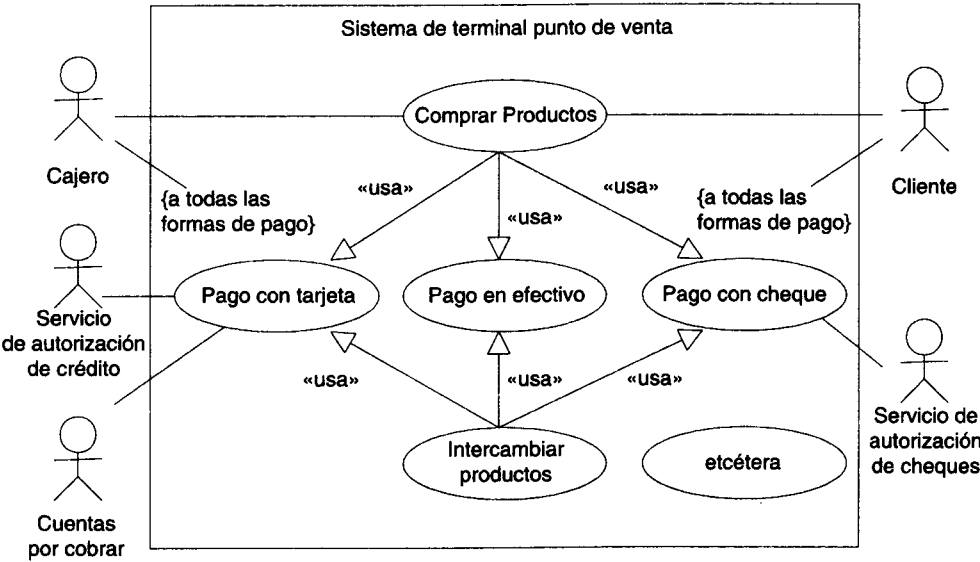


Figura 26.1 Conexión de los casos de uso por medio de la relación *usa*.

26.3.1 Ejemplo

Supongamos que los diferentes procesos de pago en el proceso *Comprar Productos* se describen en los casos de uso separados *Pago en efectivo*, *Pago con tarjeta de crédito* y *Pago con cheque*. El caso de uso *Comprar Productos* dará inicio a uno de ellos y, por lo mismo, estará en una relación *usa* con ellos.

En el lenguaje UML, la relación *usa* se indica (como se advierte en la figura 26.1) con una línea de generalización (línea continua con una flecha hueca) acompañada de la etiqueta «usa».

26.4 Documentos de casos de uso con las relaciones *usa*

Cuando los casos están en una relación *usa*, sus documentos han de expresar con palabras esa asociación. El caso que utiliza el comportamiento de otro debería indicar la conexión por medio de la palabra clave *iniciar* en el texto del caso de uso, del modo siguiente:

Caso de uso: *Comprar Productos*

...

El cliente escoge el método de pago:

- a. Si el pago se realiza en efectivo, **iniciar** *Pago en efectivo*.
- b. Si el pago se realiza con tarjeta de crédito, **iniciar** *Pago con tarjeta de crédito*.
- c. Si el pago se efectúa con cheque, **iniciar** *Pago con cheque*.

...

26.4.1 Ejemplo

Si los diferentes procesos de pago del proceso *Comprar Productos* se describen en los casos de uso aislados *Pago en efectivo*, *Pago con tarjeta de crédito* y *Pago con cheque*, el caso *Comprar Productos* iniciará uno de ellos y, por tanto, estará en una relación *usa* con ellos.

En este enfoque, los métodos de pago ya no se expresan en subsecciones del caso de uso *Comprar Productos*; son casos independientes. Nótese que podemos agregar una sección más de *Casos de uso relacionados* que resuma los casos afines, los cuales serán útiles si el caso es extenso y si no se capta fácilmente cuáles otros intervinien.

26.4.2 Comprar Productos

Caso de uso: Comprar productos
Actores: Cliente (iniciador), Cajero.
Resumen: Un cliente llega a la caja registradora con productos que desea comprar. El Cajero registra los productos y recibe el pago. Al terminar la transacción, el Cliente se marcha con los productos adquiridos.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso comienza cuando un Cliente llega a la caja registradora TPDV con productos que desea comprar.	
2. El Cajero registra los productos. Si hay más de un producto de la misma línea, también captura la cantidad.	3. Determina el precio del producto y agrega la información correspondiente a la transacción actual de ventas. Despliega la descripción y el precio del producto en cuestión.
4. Al terminar de capturar el producto, el Cajero indica a TPDV que se concluyó la venta.	5. Calcula y muestra el total de la venta.
6. El Cajero le indica el total al Cliente.	
7. El Cliente escoge la forma de pago: a. Si paga en efectivo, iniciar Pago en efectivo . b. Si paga con tarjeta de crédito, iniciar Pago con tarjeta de crédito . c. Si paga con cheque, iniciar Pago con cheque .	8. Registra la venta terminada. 9. Imprime un recibo.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
10. El Cajero entrega el recibo al Cliente.	
11. El Cliente se marcha con los productos comprados.	

Cursos alternos

- Sección 2: invalidar el identificador capturado del producto. Desplegar el error.
- Sección 7: el Cliente no pudo pagar. Cancelar la transacción de venta.

Casos de uso relacionados

- usa *Pago en efectivo*
- usa *Pago con tarjeta de crédito*
- usa *Pago con cheque*

26.4.3 Pago en efectivo

Caso de uso: Pago en efectivo
Actores: Cliente (iniciador), Cajero.
Resumen: Un Cliente paga en efectivo una compra en una terminal situada en el punto de venta.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente decide pagar en efectivo, una vez que le comunican el total de la venta.	
2. El Cliente entrega un pago en efectivo —el “efectivo ofrecido”—, posiblemente mayor que el total de la venta.	

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
3. El Cajero registra el efectivo ofrecido.	4. Muestra el cambio o vuelto que se le regresará al Cliente.
5. El Cajero deposita el efectivo recibido y extrae el dinero que devolverá.	
El Cajero entrega al Cliente el vuelto de la transacción.	

Cursos alternos

- Sección 2: el cliente no tiene suficiente efectivo. Puede cancelarse la venta o iniciarse otra forma de pago.
- Sección 3: la gaveta de la caja registradora no contiene bastante efectivo para devolver al Cliente su cambio. El Cajero pide más efectivo al supervisor o propone al Cliente otra forma de pago.

Caso de uso:	Pago con tarjeta de crédito, forma esencial.
Actores:	Cliente (iniciador), Cajero, Servicio de autorización de crédito, Cuentas por cobrar.
Resumen:	Un Cliente paga una compra con tarjeta de crédito en una terminal situada en el punto de venta. El pago es validado por un servicio externo de autorización y se registra en un sistema de cuentas por cobrar.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente decide pagar con tarjeta de crédito, una vez que le han comunicado el total de la venta.	
2. El Cliente da la información crediticia que se requiere en esta forma de pago.	3. Genera una solicitud de pago con tarjeta de crédito y la envía al Servicio externo de autorización de crédito.
4. El Servicio de autorización de crédito autoriza el pago.	5. Recibe una respuesta aprobatoria del pago por parte del Servicio de autorización de crédito (SAC).
	6. Registra el pago con tarjeta de crédito y la respuesta aprobatoria en el sistema de Cuentas por cobrar. (El servicio de autorización de crédito le debe dinero a la Tienda y, por tanto, cuentas por cobrar debe darle seguimiento.)
	7. Muestra el mensaje con que se concede la autorización.

Cursos alternos

- Sección 4: el Servicio de autorización de crédito rechaza la solicitud de crédito. Proponerle al Cliente otra forma de pago.

Caso de uso:	Pago con cheque, forma esencial.
Actores:	Cliente (iniciador), Cajero, Servicio de autorización de cheques.
Resumen:	Un Cliente paga con cheque una compra en una terminal situada en el punto de venta. El pago es validado por un servicio externo de autorización de cheques.

26.4.5 Pago con cheque

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente decide pagar con cheque, una vez que le han comunicado el total de la venta.	
2. El Cliente extiende un cheque y presenta una identificación.	
3. El Cajero registra la información relativa a la identificación y solicita la autorización del pago con cheque.	4. Genera una solicitud de pago con cheque y la envía al Servicio externo de autorización de cheques.
5. El Servicio de autorización de cheques autoriza el pago.	6. Recibe la respuesta aprobatoria del pago por parte del Servicio de autorización de cheques.
	7. Muestra el mensaje de que se obtuvo la autorización.

Cursos alternos

- Sección 5: solicitud de pago con cheque rechazada por el Servicio de autorización. Sugerirle al Cliente otra forma de pago.

EXTENSIÓN DEL MODELO CONCEPTUAL

Objetivos

- Identificar nuevos conceptos para incluirlos en el segundo ciclo de desarrollo.

27.1 Nuevos conceptos en el sistema del punto de venta

Igual que en el primer ciclo de desarrollo, podemos desarrollar poco a poco el modelo conceptual considerando los casos actuales de uso en el segundo ciclo. Primero repasamos la *lista de categorías de conceptos* y luego aplicamos la identificación de frases nominales o sustantivos a partir de los casos de uso (con la advertencia habitual de que este segundo método exige mucha sensatez y prudencia). Como vimos en capítulos anteriores, un procedimiento muy eficaz para crear un modelo conceptual robusto y rico consiste en estudiar el trabajo que otros autores han dedicado al tema, sobre todo los que se valen del estudio de casos, entre ellos [Fowler96]. En este libro no vamos a examinar la multitud de cuestiones tan sutiles concernientes a la construcción de modelos que analizan en sus obras.

Nuestras estrategias fundamentales no darán por resultado el modelo conceptual más útil. En este capítulo elaboraremos un modelo provisional, que nos sirva simplemente para descubrir los conceptos iniciales. En capítulos posteriores redondearemos los detalles a partir de otros conceptos, atributos y asociaciones con el propósito de desarrollar un mejor modelo.

27.1.1 Lista de las categorías de los conceptos

La siguiente tabla tan sólo contiene los conceptos nuevos en el problema del sistema del punto de venta.

Categoría	Ejemplos
objetos físicos o tangibles	TarjetadeCredito, Cheque
especificaciones, diseños o descripciones de cosas	
lugares	
transacciones	PagoenEfectivo, PagoconTarjetadeCredito, PagoconCheque
transacciones con líneas de producto	
papeles de las personas	
contenedores de otras cosas	
cosas dentro de un contenedor	
otros sistemas de cómputo o electromecánicos externos a nuestro sistema	ServiciodeAutorizaciondeCredito, ServiociodeAutorizaciondeCheques
conceptos sustantivos abstractos	
organizaciones	ServiciodeAutorizaciondeCredito, ServiociodeAutorizaciondeCheques
eventos	
reglas y políticas	
catálogos	
registros de finanzas, de trabajo, de contratos, de asuntos legales	CuentasporCobrar
instrumentos y servicios financieros	
manuales, libros	

27.1.2 Identificación de las frases nominales en los casos de uso

La identificación de las frases nominales o nombres sustantivos, repetimos, no puede aplicarse mecánicamente para descubrir los conceptos pertinentes que incluiremos en

un modelo conceptual. Es necesario recurrir al sentido común y a las abstracciones idóneas, pues el lenguaje natural es ambiguo y los conceptos relevantes no siempre se expresan de un modo explícito o claro en el texto. No obstante, es un procedimiento práctico en la construcción de modelos conceptuales por su gran simplicidad.

En teoría, recomendamos emplear los casos *reales* de uso —y no los casos de uso esenciales— en la identificación de conceptos, porque contienen especificaciones más detalladas y concretas.

Pago en efectivo

No hay nuevos conceptos, pues los que se utilizan ya se explicaron en la iteración 1. Pese a ello, podemos especificar el nombre del pago como *PagoenEfectivo*.

Pago con tarjeta de crédito

1. Este caso de uso comienza cuando un Cliente decide pagar con tarjeta de crédito, una vez que le han comunicado el total de la venta.

2. El Cliente desliza su **tarjeta de crédito** en un lector de tarjetas para efectuar el **pago con tarjeta de crédito**.
3. Genera una **solicitud de pago con tarjeta de crédito** y la envía a un **Servicio externo de autorización de crédito**, a través de un módem conectado a la TPDV. Requiere que se marque al servicio, que se transmita un registro de solicitud y que se espere el registro de respuesta.

4. Recibe una **respuesta aprobatoria de crédito por parte** del Servicio de autorización de crédito (SAC). Se codifica la respuesta en un registro de respuesta y se recibe a través del módem.

5. Registra en el sistema **Cuentas por cobrar** el pago con tarjeta y la información de la respuesta aprobatoria. (El Servicio externo de autorización de crédito le debe dinero a la Tienda y por eso Cuentas por cobrar debe darle seguimiento.)

6. Muestra el mensaje de autorización del crédito.

Pago con cheque

1. Este caso de uso comienza cuando un Cliente decide pagar con cheque, una vez que le han comunicado el total de la venta.

2. El Cliente extiende un **cheque** y se lo entrega al Cajero junto con su **licencia de conducir**.

3. El Cajero anota en el cheque el **número de la licencia de conducir**, lo teclea y lo introduce en el campo texto del *Número LC* en la ventana; después oprime el botón *Autorización de Cheques* para solicitar la autorización del pago con cheque.
4. Genera una **solicitud de pago con cheque** y la envía a un **Servicio externo de autorización de cheques** a través de un **módem** conectado a la TPDV. Se requiere telefonear al servicio, transmitir un registro de solicitud, y esperar el registro de respuesta.

5. Recibe una **respuesta aprobatoria del cheque** por parte del Servicio de autorización de cheques. Se codifica la respuesta en un registro de respuesta y se recibe a través del módem.

6. Muestra el mensaje de autorización del pago.

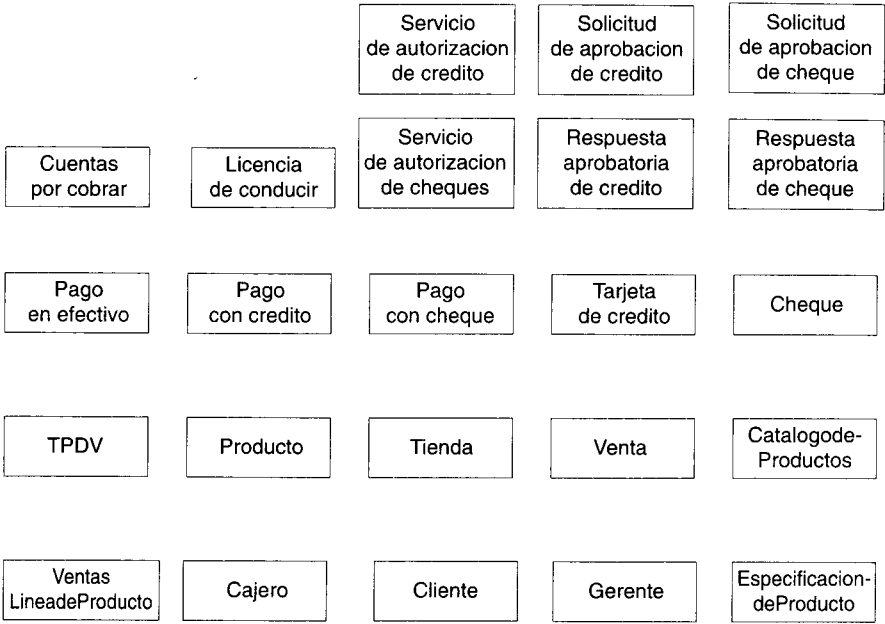


Figura 27.1 Modelo conceptual preliminar para el dominio de la terminal punto de venta.

27.1.3 Transacciones con el servicio de autorización

La identificación de frases nominales en los casos de uso revela conceptos como *SolicituddePagoconTarjeta* y *RespuestaAprobatoriadePagoconTarjeta*. Es posible ver estos conceptos como tipos de transacciones con los servicios externos; en términos generales, conviene identificarlas porque la actividad y los procesos suelen girar en torno a ellas.

Estas transacciones no necesariamente deben representar registros de computadora ni bits que se desplacen sobre una línea. Representan la abstracción de la transacción independientemente del medio con que se ejecute. Así, una solicitud de pago con tarjeta de crédito puede ser ejecutada por personas que se comunican con teléfono, por dos computadoras que se transmiten registros o mensajes y por otros medios.

27.1.4 El modelo conceptual TPDV: bosquejo 1

En la figura 27.1 se incluye el modelo conceptual preliminar que se basa en la identificación de conceptos mediante la lista de comprobación y la identificación de sustantivos. No es tan útil ni tan completo como lo hubiéramos deseado; por eso, en los capítulos subsecuentes vamos a examinar detenidamente otras listas concernientes a la construcción de modelos.

GENERALIZACIÓN

Objetivos

- Crear jerarquías de generalización-especialización.
- Identificar cuándo vale la pena mostrar un subtipo.
- Aplicar las reglas de 100% y de Es-un para validar los subtipos.

28.1 Generalización

Los conceptos de *Pago en Efectivo*, *Pago con Tarjeta* y *Pago con Cheque* se parecen mucho. En este caso es posible (y útil)¹ organizarlos (figura 28.1) en una **jerarquía de tipo generalización-especialización** (o simplemente en una **jerarquía de tipo**) en la cual el **supertipo** *Pago* represente un concepto más general y los **subtipos**, conceptos más especializados.

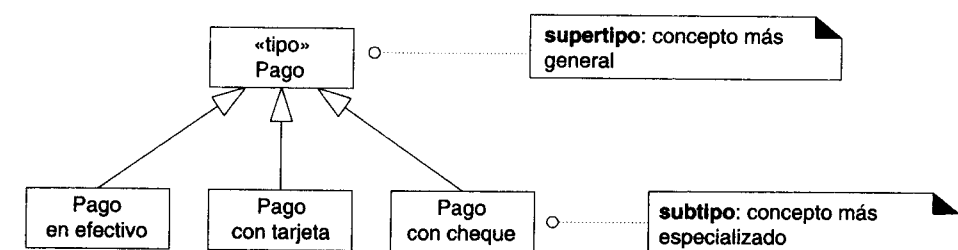


Figura 28.1 Jerarquía de generalizaciones-especificaciones.

¹ Más adelante, en este mismo capítulo, investigaremos las razones por las cuales se definen las jerarquías de tipos.

La **generalización** es la actividad consistente en identificar los aspectos comunes de los conceptos y en definir las relaciones entre el supertipo (concepto general) y el subtipo (concepto especializado). Es una forma de efectuar clasificaciones taxonómicas entre los conceptos que después se ejemplifican en las jerarquías de tipos.

Identificar los supertipos y los subtipos es una labor útil en un modelo conceptual, porque su presencia nos permite entender los conceptos en términos más generales, más complejos y más abstractos. Favorece además la simplificación de la expresión, mejora la comprensión y reduce la información redundante. Y aunque por ahora nos concentraremos en un modelo conceptual y no en una implementación de software, obtenemos mejores programas de cómputo al implementar después los supertipos y los subtipos como clases.

Por tanto:

Identifique los supertipos y subtipos del dominio que se relacionan con la investigación en curso y descríbalos gráficamente en el modelo conceptual.

28.1.1 Notación de UML

En el lenguaje UML, la relación de generalización entre los elementos se indica con una punta de flecha grande y hueca que señala el elemento más general partiendo del más especializado. Puede emplearse un estilo de flechas blanco separadas o de una flecha compartida.

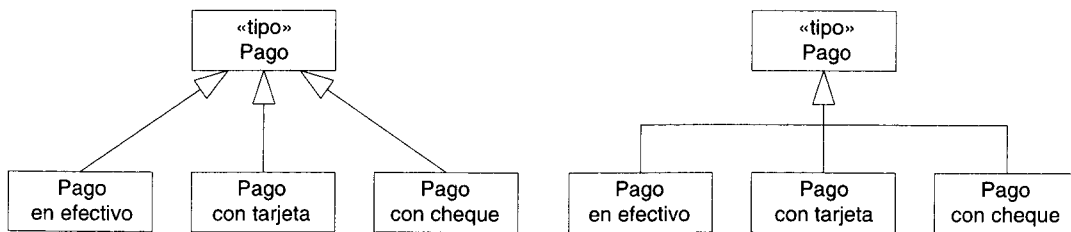


Figura 28.2 Jerarquía de tipos con notación de flechas separadas y con notación de una flecha compartida.

28.2 Definición de supertipos y de subtipos

Por ser de gran utilidad identificar los supertipos y los subtipos, conviene conocer con toda claridad y precisión la generalización, los supertipos y los subtipos a partir de la definición y de los conjuntos de tipos.¹ En la siguiente sección estudiamos los tipos.

¹ En otras palabras, la denotación y connotación de un tipo. Esta explicación se inspira en [MO95].

28.2.1 Generalización y la definición de tipos

¿Qué relación existe entre un supertipo y un subtipo?

La definición de un supertipo es más general o amplia que la de un subtipo.

Consideremos, por ejemplo, el supertipo *Pago* y sus subtipos (*Pago en Efectivo* y otros). Supongamos que, según su definición, *Pago* representa la transacción de transferir dinero (no necesariamente efectivo) de una entidad a otra en una compra y que en todos los pagos se refiere cierta cantidad de dinero. El modelo correspondiente a esta transacción aparece en la figura 28.3.

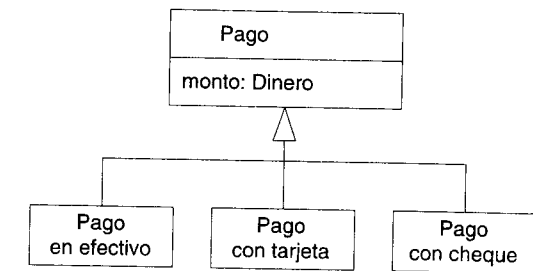


Figura 28.3 Jerarquía de los tipos de pago.

Un *Pago con Tarjeta* es una transferencia de dinero que se realiza a través de una institución de crédito y que ha de ser autorizada. Mi definición de *Pago* abarca la de *Pago con Tarjeta* y es más general que ella.

28.2.2 Generalización y los conjuntos de tipos

Los subtipos y los supertipos están relacionados por su pertenencia a un conjunto.

Todos los miembros de un conjunto de subtipo pertenecen al respectivo conjunto de supertipo.

Por ejemplo, en lo tocante a la pertenencia a un conjunto, todas las instancias del conjunto *Pago con Tarjeta* también son miembros del conjunto de *Pago*. Esto se muestra en la figura 28.4 como un diagrama de Venn.

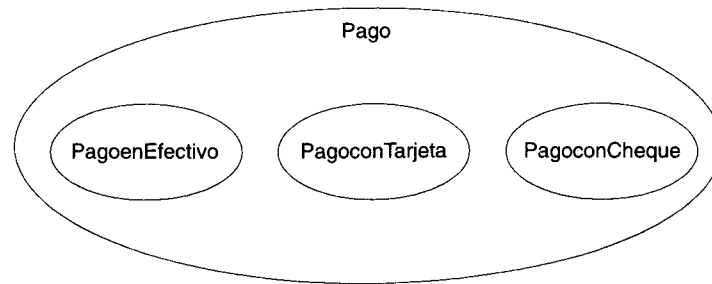


Figura 28.4 Diagrama de Venn para las relaciones de un conjunto.

28.2.3 Conformidad con la definición de subtipos

Cuando se crea una jerarquía de tipos, en torno a los supertipos se hacen afirmaciones que se aplican también a los subtipos. Así, la figura 28.5 señala que todos los *Pagos* tienen un *monto* y que se asocian a una *Venta*.

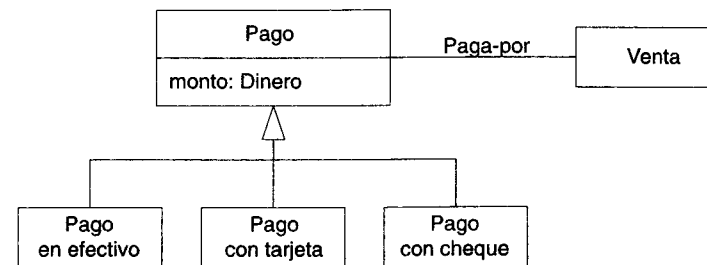
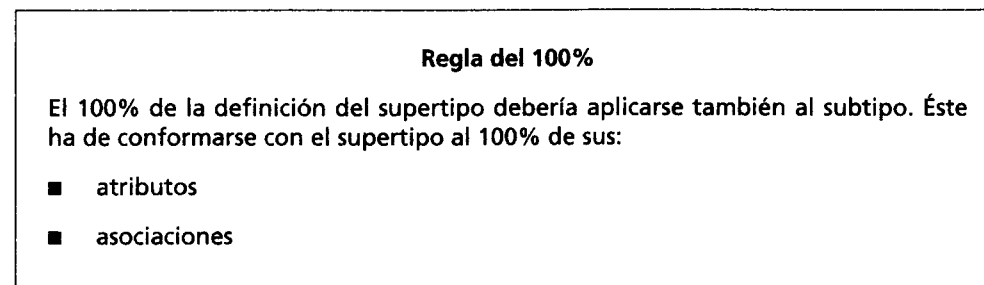


Figura 28.5 Conformidad con los subtipos.

Todos los subtipos de *Pago* han de conformarse y tener un monto y un pago de una *Venta*. En términos generales, a esta regla de conformidad con una definición de un supertipo se le da el nombre de *Regla de 100%*:

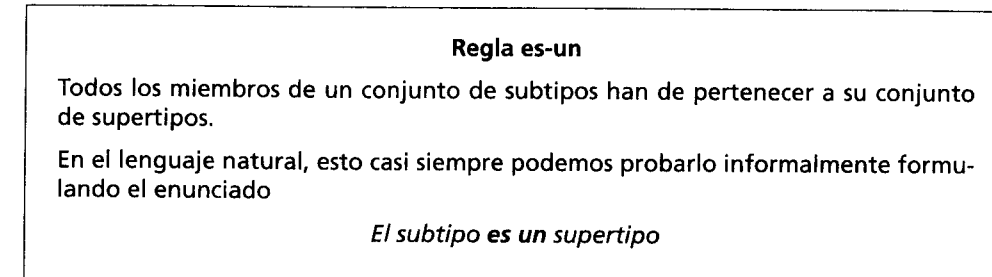


28.2.4 Conformidad con el conjunto de subtipos

Un subtipo debería ser miembro del conjunto del supertipo. Así, *PagoconTarjeta* debería pertenecer al conjunto de *Pagos*.

De un modo informal, la afirmación anterior expresa la idea de que el subtipo *es una clase de* supertipo. *PagoconTarjeta es una clase de Pago*. En un lenguaje más técnico, *es-una-clase-de* que se designa como *es-un*.

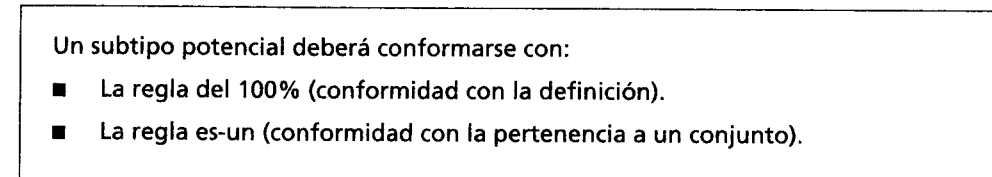
Este tipo de conformidad es la *Regla es-un*:



Por ejemplo, el enunciado el *PagoconTarjeta es un Pago* tiene sentido y expresa la idea de conformidad con la pertenencia a un conjunto.

28.2.5 ¿Qué es un subtipo correcto?

Basándose en la explicación que acabamos de ofrecer, aplique las siguientes pruebas¹ para definir un subtipo correcto cuando construya un modelo conceptual:



28.3 Cuándo definir un subtipo

Hemos expuesto las reglas que garantizan la corrección de un subtipo (la regla del 100% y la regla es-un). Pero, ¿cuándo deberíamos tomarnos la molestia de definir un subtipo? Ante todo, una definición: una **partición de tipo** es una división del tipo en subtipos disjuntos [MO95].

¹ Los nombres de estas reglas se eligieron (en inglés) por su utilidad mnemotécnica más que por su precisión.

La pregunta también puede plantearse así: "¿cuándo conviene mostrar una partición de tipos?"

Por ejemplo, en el dominio del punto de venta, *Cliente* puede partirse (o dividirse en subtipos) correctamente en *ClienteVarón* y en *ClienteMujer*. ¿Pero es relevante o útil incluir esto en nuestro modelo (figura 28.6)?

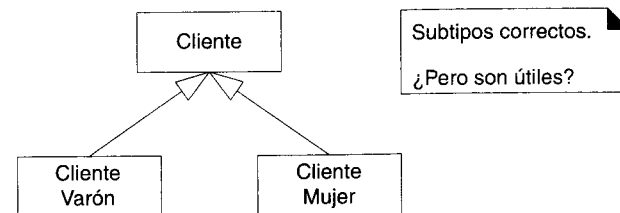


Figura 28.6 Particiones de tipos legales, ¿pero son útiles en nuestro dominio?

Esta partición no es útil en nuestro dominio; en la siguiente sección explicamos el porqué.

28.3.1 Motivos para partir un tipo en subtipos

A continuación se enumeran algunos motivos muy convincentes para partir un tipo en subtipos.

Se crea un subtipo de un supertipo cuando:

1. El subtipo tiene otros atributos más de interés.
2. El subtipo tiene otras asociaciones más de interés.
3. Se opera sobre el concepto del subtipo, se maneja, se reacciona ante él o se manipula de modo diferente a como se haría con el supertipo u otros subtipos, todo ello en aspectos que resultan relevantes.
4. El concepto de subtipo representa una cosa animada (un robot o un animal, por ejemplo) que se comporta de manera distinta al supertipo o a otro subtipo en aspectos que resultan relevantes.

Conforme a los criterios anteriores, no es obligatorio partir *Cliente* en *ClienteVarón* y *ClienteMujer* porque no poseen atributos ni asociaciones adicionales, no se opera sobre ellos (ni se les trata) en forma diferente y tampoco muestran un comportamiento distinto que resulte de interés.¹

¹ Los hábitos de compra de los hombres son distintos a los de las mujeres. Pero tales diferencias no son importantes para los requerimientos de nuestro caso de uso, lo cual constituye el criterio que rige nuestra investigación.

A continuación se incluyen algunos ejemplos de partición de tipos en el dominio de los pagos y en otras áreas, aplicando los siguientes criterios:

Justificación del subtipo	Ejemplos
El subtipo tiene otros atributos más de interés.	Pagos: no aplicable. Biblioteca: <i>Libro</i> , subtipo de <i>Recurso-Prestable</i> , tiene un atributo de <i>ISBN</i> .
El subtipo tiene otras asociaciones más de interés.	Pagos: <i>PagoconTarjeta</i> , subtipo de <i>Pago</i> , se asocia a <i>TarjetadeCredito</i> . Biblioteca: <i>Video</i> , subtipo de <i>Recurso-Prestable</i> , se asocia a <i>Director</i> .
Se opera sobre el concepto del subtipo, se maneja, se reacciona ante él o se manipula en forma diferente a como se haría con el supertipo u otros subtipos, todo ello en aspectos que resultan de interés.	Pagos: <i>PagoconTarjeta</i> , subtipo de <i>Pago</i> , se maneja en forma distinta a otros tipos de pago en lo tocante a la forma de autorizarse. Biblioteca: <i>Software</i> , subtipo de <i>RecursoPrestable</i> , requiere un depósito para que se preste.
El concepto de subtipo representa un ser animado (un robot o un animal, por ejemplo) cuyo comportamiento difiere del de otro supertipo o subtipos en aspectos que resultan de interés.	Pagos: no aplicable. Biblioteca: no aplicable. Investigación de mercado: <i>Humano-Varon</i> , subtipo de <i>Humano</i> , no se comporta igual que <i>HumanoMujer</i> respecto a los hábitos de compra.

28.4 Cuándo definir un supertipo

Se acostumbra convertir la generalización en un supertipo común cuando se descubren aspectos comunes entre los subtipos potenciales.

A continuación se enumeran algunos motivos que hacen generalizar y definir un supertipo.

Se crea un supertipo en una relación de generalización con subtipos cuando:

- Los subtipos potenciales representan variaciones de un concepto semejante.
- Los subtipos se conforman a las reglas del 100% y de es-un.
- Todos los subtipos con un mismo atributo pueden factorizarse y expresarse en el supertipo.
- Todos los subtipos tienen la misma asociación que puede factorizarse y relacionarse con el supertipo.

En las siguientes secciones los puntos anteriores se aclaran por medio de ejemplos.

28.5 Jerarquías de los tipos del punto de ventas

28.5.1 Tipos de pago

Conforme a los criterios precedentes con que se parte el tipo *Pago*, conviene elaborar una jerarquía de tipos de varias clases de pagos. El motivo de crear supertipos y subtipos se indica en la figura 28.7.

28.5.2 Tipos del servicio de autorización

Los servicios de autorización de créditos y cheques son variantes de un concepto similar y presentan atributos comunes de interés. Se obtiene así la jerarquía de tipos de la figura 28.8.

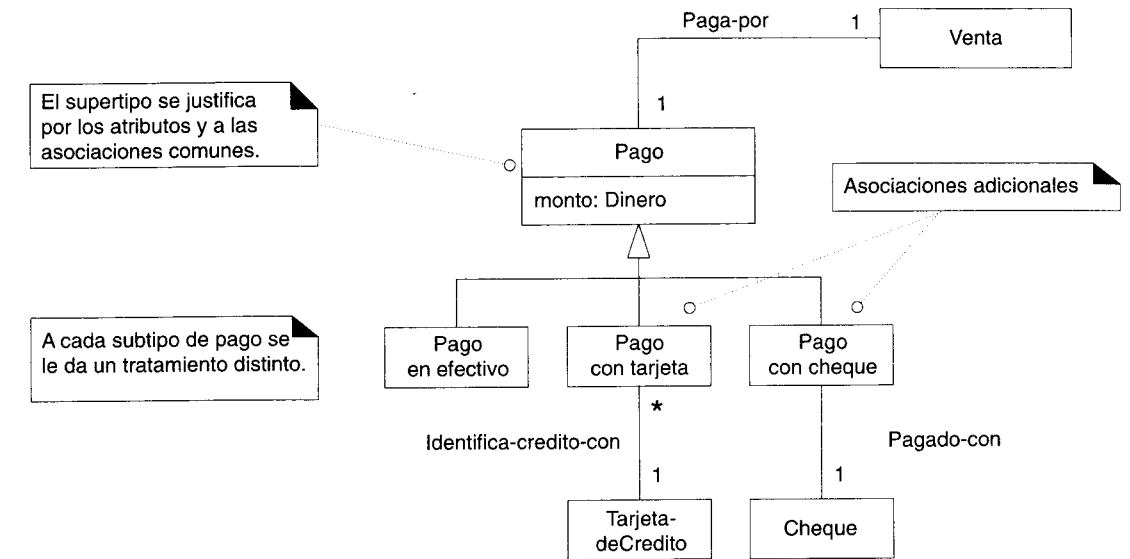


Figura 28.7 Justificación de los subtipos de *Pago*.

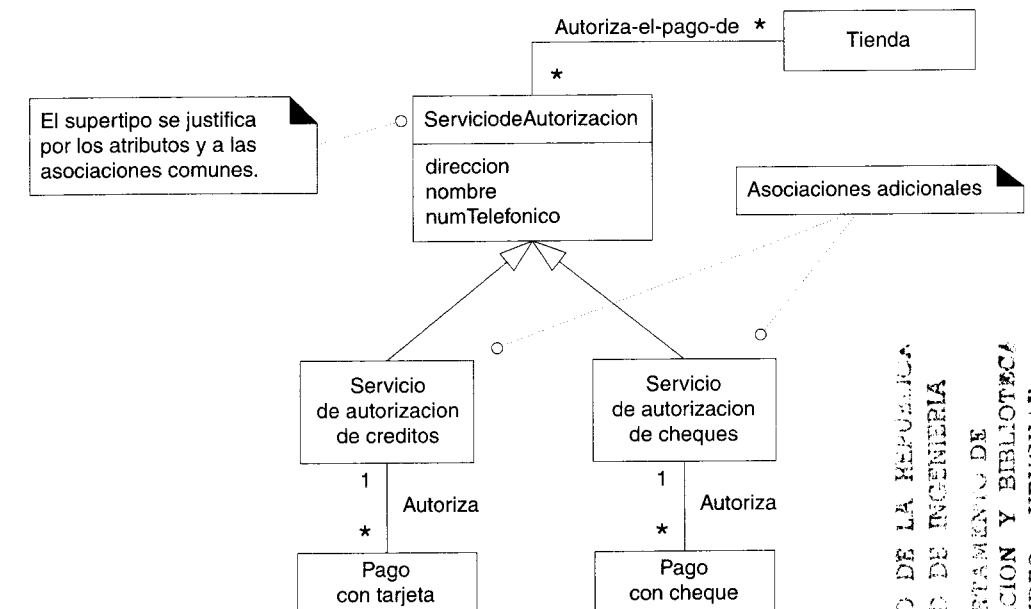


Figura 28.8 Justificación de la jerarquía *ServiciodeAutorizacion*.

28.5.3 Tipos de transacciones de autorización

El modelado de varios tipos de transacciones (solicitudes y respuestas) de los servicios de autorización presenta un caso interesante. Por lo regular las transacciones con los servicios externos sirven para mostrar un modelo conceptual, porque la actividad y los procesos suelen girar en torno a ellas. Se, trata, pues, de conceptos importantes en extremo.

¿Debería el modelador dar ejemplos de *todas* las variantes de una transacción con algún servicio externo? No podemos ofrecer una respuesta válida para todos los casos. Según mencionamos con anterioridad, los modelos conceptuales no son intrínsecamente correctos ni erróneos, sino de utilidad variable. Esto debemos tenerlo presente, porque cada tipo de transacción se relaciona con distintos conceptos, procesos y reglas de negocios.¹

Un segundo punto de mucho interés es el grado de generalización cuya utilidad respalde su inclusión en el modelo. Para facilitar nuestra explicación supongamos que toda transacción tiene fecha y hora. Estos atributos comunes, junto con el deseo de formular una generalización fundamental para esta familia de conceptos relacionados, justifica la creación de la *TransacciondeAutorizaciondePago*.

¿Pero conviene generalizar una respuesta e incorporarla a *RespuestadeAutorizaciondeCredito* y *RespuestadeAutorizaciondePagoconCheque*, como se advierte en la figura 28.9, o basta para mostrar una menor generalización, como se indica en la figura 28.10?

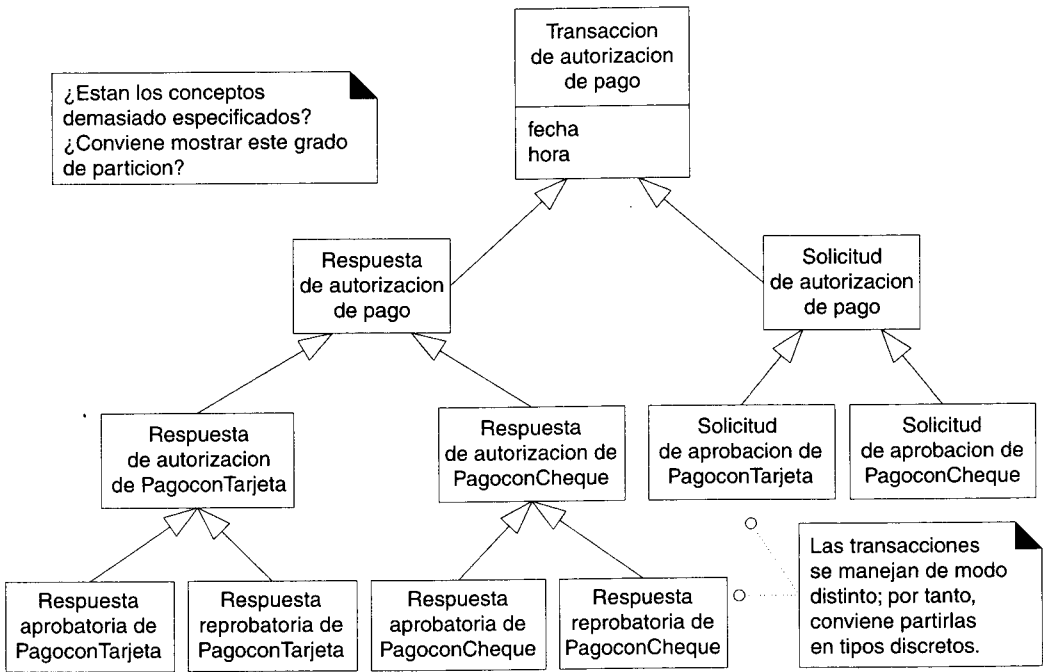


Figura 28.9 Una jerarquía posible de tipos para las transacciones con un servicio externo.

¹ En los modelos conceptuales de telecomunicaciones, también recomendamos identificar cada tipo de mensaje de cambio o intercambio.

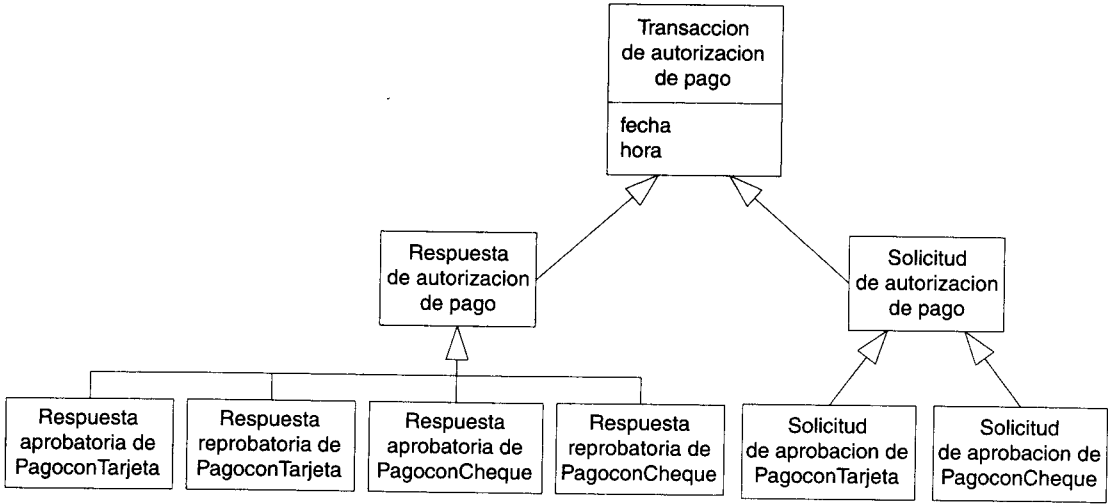


Figura 28.10 Jerarquía alterna de tipos de transacción.

El tipo de jerarquía de la figura 28.10 es suficientemente útil en lo tocante a la generalización, porque las generalizaciones complementarias no aportan un valor evidente. La jerarquía de la figura 28.9 expresa una granularidad más fina de generalización, la cual no mejora significativamente nuestro conocimiento de los conceptos ni de las reglas de negocios; pero sí hace más intrincado el modelo, además de que no conviene una mayor complejidad salvo que aporte otros beneficios.

28.6 Tipos abstractos

En la presente sección definiremos la designación del tipo abstracto. Se recomienda identificar esta clase de tipos en un modelo conceptual porque limitan de cuáles tipos es posible dar instancias concretas, aclarando con ello las reglas del dominio del problema.

Si todos los miembros de un tipo T han de serlo también de un subtipo, al tipo T se le da el nombre de **tipo abstracto**.

Supongamos, por ejemplo, que todas las instancias *Pago* han de ser más específicamente una instancia del subtipo *PagoconTarjeta*, *PagoenEfectivo* o *PagoconCheque*. Esto se muestra gráficamente con el diagrama de Venn de la figura 28.11 (b). Dado que todo miembro *Pago* pertenece también a un subtipo, *Pago* será un tipo abstracto por definición.

En cambio, si puede haber instancias *Pago* que no sean miembro de un subtipo, no será un tipo abstracto según se aprecia en la figura 28.11 (a).

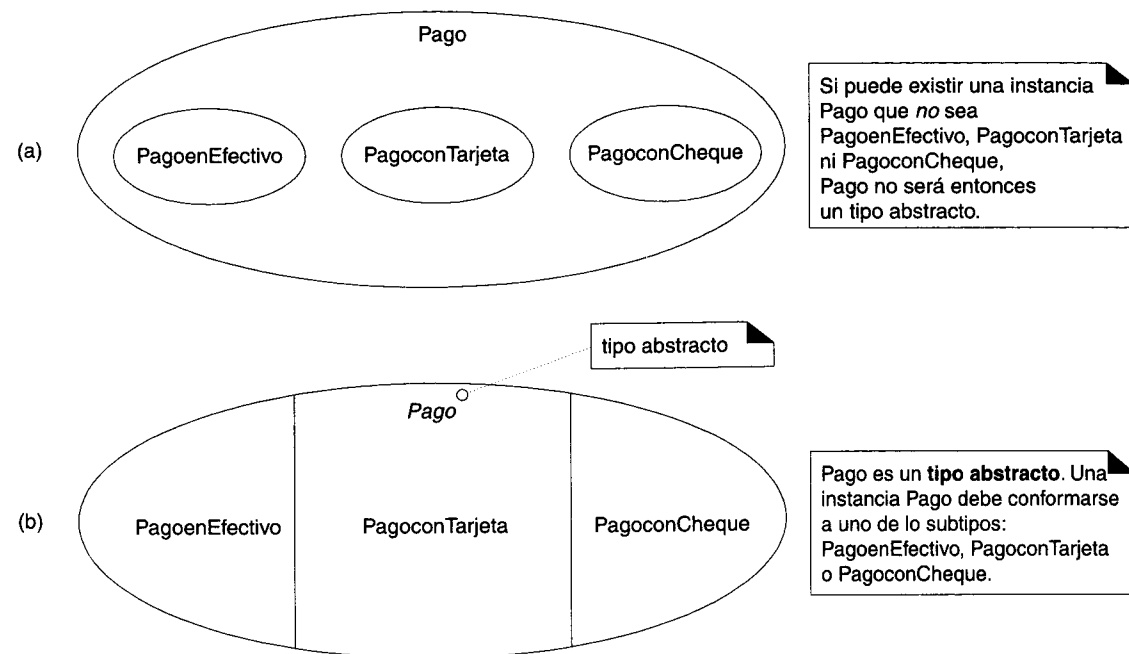
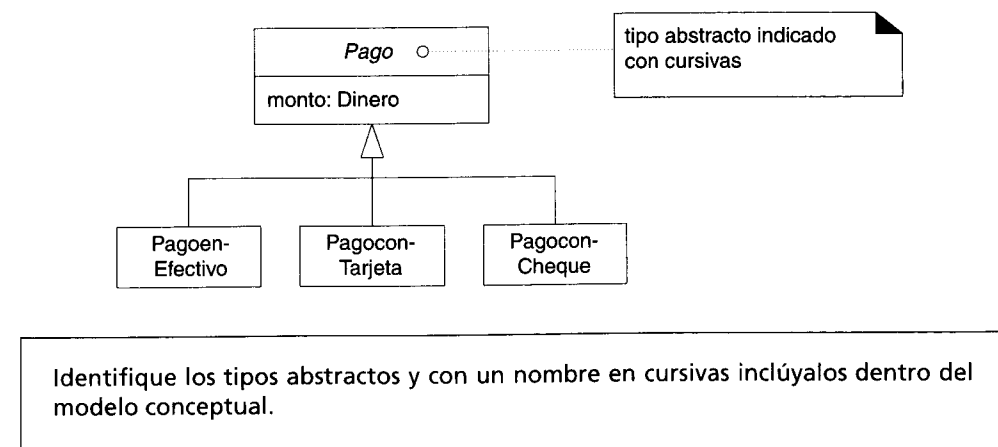


Figura 28.11 Tipos abstractos.

En el dominio del punto de venta, todo *Pago* es en realidad miembro de un subtipo. La figura 28.11 (b) es la descripción exacta de los pagos y, por tanto, *Pago* es un tipo abstracto.

28.6.1 Notación de tipos abstractos en el lenguaje UML

El UML cuenta con una notación que indica los tipos abstractos, y el nombre de tipo se imprime en cursivas.



28.6.2 Clases y métodos abstractos

Si un tipo abstracto se implementa en el software como una clase durante la fase de diseño, solemos representarlo con una **clase abstracta**, lo cual significa que no podemos crear instancias para ella. Un **método abstracto** es aquel que se declara en una clase abstracta, pero que no se implementa en ella; en el lenguaje UML se escribe también en cursivas.

28.7 Construcción de modelos con estados cambiantes

Suponga que un pago puede estar en estado autorizado o no autorizado, y que podemos presentar esos dos estados en el modelo conceptual (tal vez no sea así, pero lo supondremos en nuestra exposición). Como se observa en la figura 28.12, una forma de construir el modelo respectivo consiste en definir los subtipos de *Pago*: *PagoNoAutorizado* y *PagoAutorizado*. Pero adviértase que un pago no permanece en uno de estos estados; ordinariamente se da una transición del estado no autorizado al autorizado. De aquí deducimos la siguiente directriz:

No modele los estados de un concepto X como subtipos de X. Por el contrario:

1. Defina una jerarquía de estados y asóciela a X.
2. No incluya los estados de un concepto en el modelo conceptual; inclúyalos en los diagramas de estado.

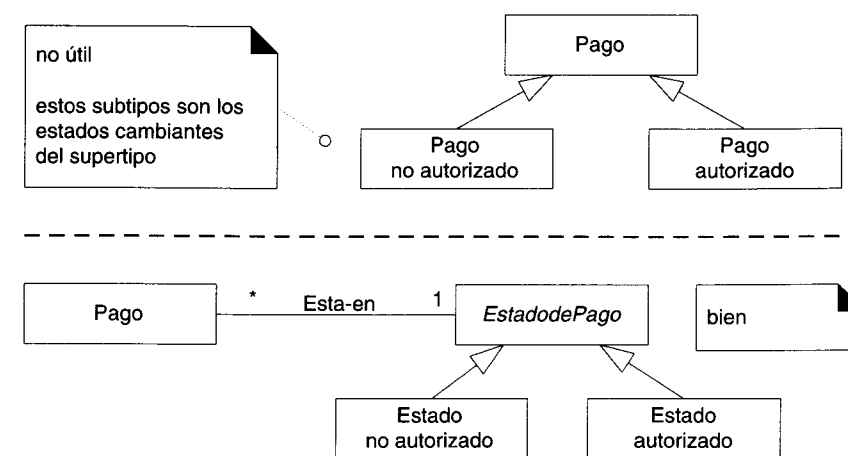


Figura 28.12 Modelado de los estados cambiantes.

28.8 Jerarquías de clases y herencia

En nuestra exposición sobre las jerarquías de tipos no hemos mencionado la *herencia*, porque nos hemos centrado en un modelo conceptual de entidades del mundo real, no en artefactos de software. Una clase es una implementación de un concepto o de un tipo en software, y en un lenguaje de programación orientado a objetos una subclase **hereda** las definiciones de atributos y de operaciones de sus superclases, gracias a la creación de **jerarquías de clases**. La **herencia** es un mecanismo de software que implementa la conformidad de los subtipos con sus definiciones de supertipo. Por tanto, la herencia no interviene en la explicación de un modelo conceptual, pero sí lo hace cuando efectuamos la transición a la fase de diseño.

Las jerarquías de tipos aquí generadas pueden reflejarse o no en nuestra solución. Así, es posible reducir o ampliar la jerarquía de los tipos de transacciones relacionadas con los servicios de autorización, integrándolas a las jerarquías de clases de software, según las características del lenguaje y otros factores. Por ejemplo, las clases de plantillas de C++ podrían servirnos para reducir el número de clases por desarrollar.

PAQUETES: ORGANIZACIÓN
DE LOS ELEMENTOS

Objetivos

- Organizar los elementos en paquetes.

29.1 Introducción

El modelo conceptual del segundo ciclo de desarrollo para la aplicación del punto de venta comienza a tener un tamaño excesivo; advertimos entonces la necesidad de dividir sus elementos en subconjuntos más pequeños.

Organizar los elementos en paquetes ofrece la ventaja de separar los elementos detallados en abstracciones más amplias, lo cual brinda soporte a una vista de nivel superior y permite contemplar el modelo en agrupamientos más simples.

En este capítulo examinaremos cómo se utilizan los paquetes para particionar el modelo conceptual.

29.1.1 Capas y particiones

Según señalamos antes, podemos concebir la arquitectura global del sistema como si estuviera compuesta de particiones verticales y horizontales. Los paquetes del modelo conceptual, en caso de usarse en el diseño, pueden considerarse *particiones* de la capa de objetos del dominio.

29.2 Notación de los paquetes en el UML

Un paquete se muestra gráficamente como una carpeta etiquetada (figura 29.1). En su interior podemos incluir los paquetes subordinados. El nombre del paquete se halla dentro de la etiqueta si el paquete describe sus elementos; de lo contrario aparecerá en el centro de ella.

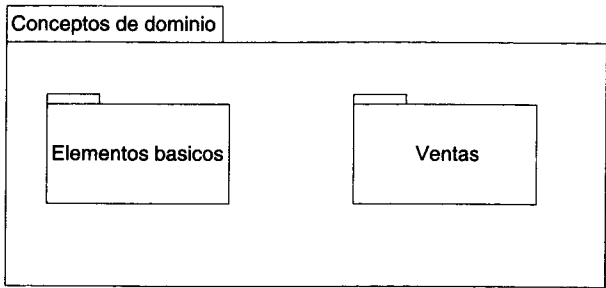


Figura 29.1 Un paquete representado con el lenguaje UML.

29.2.1 Propiedad y referencias

Un elemento —un tipo o una clase, por ejemplo— es *propiedad* del paquete dentro del cual está definido, pero puede *referenciarse* en otros. En tal caso su nombre se especifica con el del paquete que emplea el formato del nombre de la trayectoria *Nombre del Paquete::Nombre del Elemento* (figura 29.2). El tipo o clase incluidos en un paquete extraño pueden modificarse con nuevas asociaciones, pero por lo demás han de permanecer inalterados.

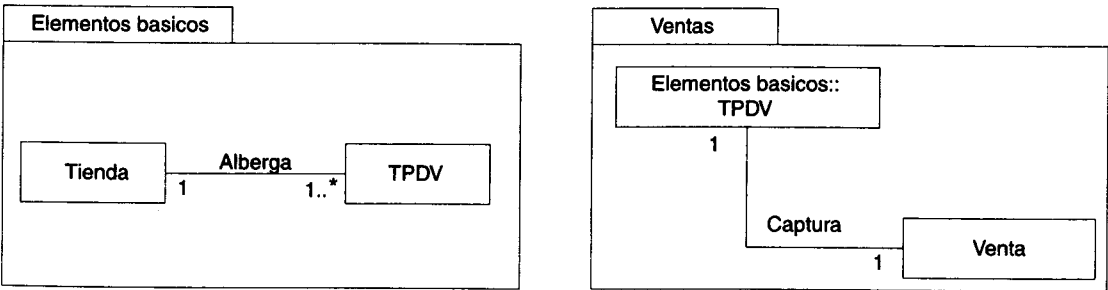


Figura 29.2 Tipo referenciado en un paquete.

29.2.2 Dependencias de los paquetes

Si de alguna manera un elemento de un modelo depende de otro, la dependencia puede mostrarse con una relación de subordinación, que indica una línea con flecha.

La dependencia de un paquete significa que sus elementos conocen en alguna forma los del paquete objetivo y están acoplados a ellos. Por ejemplo, existirá dependencia si un paquete referencia un elemento que es propiedad de otro. Así, el paquete *Ventas* tiene dependencia respecto al de *Elementos Basicos* (figura 29.3).

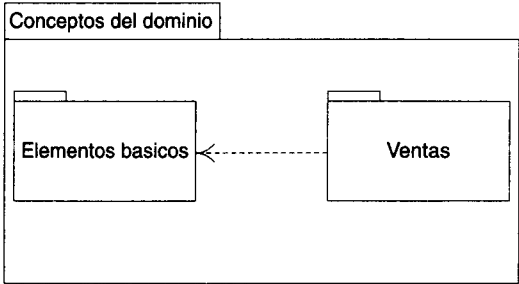


Figura 29.3 Una dependencia de paquete.

29.2.3 Indicación de paquetes sin diagrama

A veces no conviene dibujar un diagrama de paquete, pero sí indicar el paquete del cual forman parte los elementos.

En tal caso se incluye una nota de restricción (nota con una esquina doblada) en el diagrama, como se aprecia en la figura 29.4.

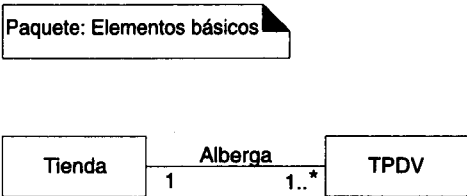


Figura 29.4 Presentación gráfica de la propiedad de un paquete con restricción.

29.3 Cómo partir el modelo conceptual

En esta sección nos centraremos en la partición del modelo conceptual; en el capítulo 22 explicaremos la organización de la arquitectura del software en paquetes.

¿Cómo debemos organizar en paquetes los tipos de un modelo conceptual?

La respuesta es sencilla: basta aplicar las siguientes directrices generales:

Para partir el modelo conceptual en paquetes, reúna los elementos que:

- se encuentren en la misma área o tema, o sea que estén estrechamente relacionados por un concepto o propósito
- se encuentren en la misma jerarquía de tipos
- participen en los mismos casos de uso
- presenten una asociación muy íntima.

Conviene que todos los elementos relacionados con el modelo conceptual estén contenidos en un paquete denominado *Conceptos del Dominio* y que los conceptos comunes compartidos ampliamente se definan con un nombre empaquetado como *Elementos Básicos* o *Conceptos Comunes*, cuando no haya un paquete significativo donde incluirlos.

29.4 Paquetes del modelo conceptual del punto de venta

Con base en los criterios anteriores, en la figura 29.5 se ofrece la organización de un paquete para el dominio del punto de venta.

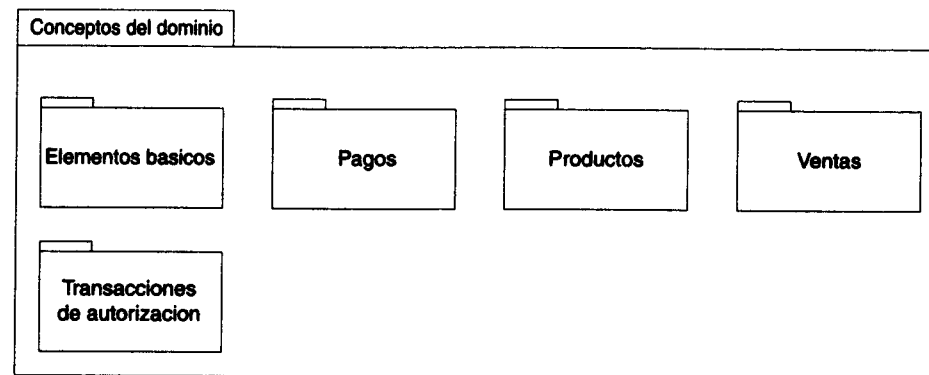


Figura 29.5 Paquetes de conceptos del dominio.

Los diagramas de paquete de las figuras 29.6 y 29.7 describen gráficamente la propiedad de los tipos individuales. En los siguientes capítulos vamos a examinar los detalles de las asociaciones y de los atributos.

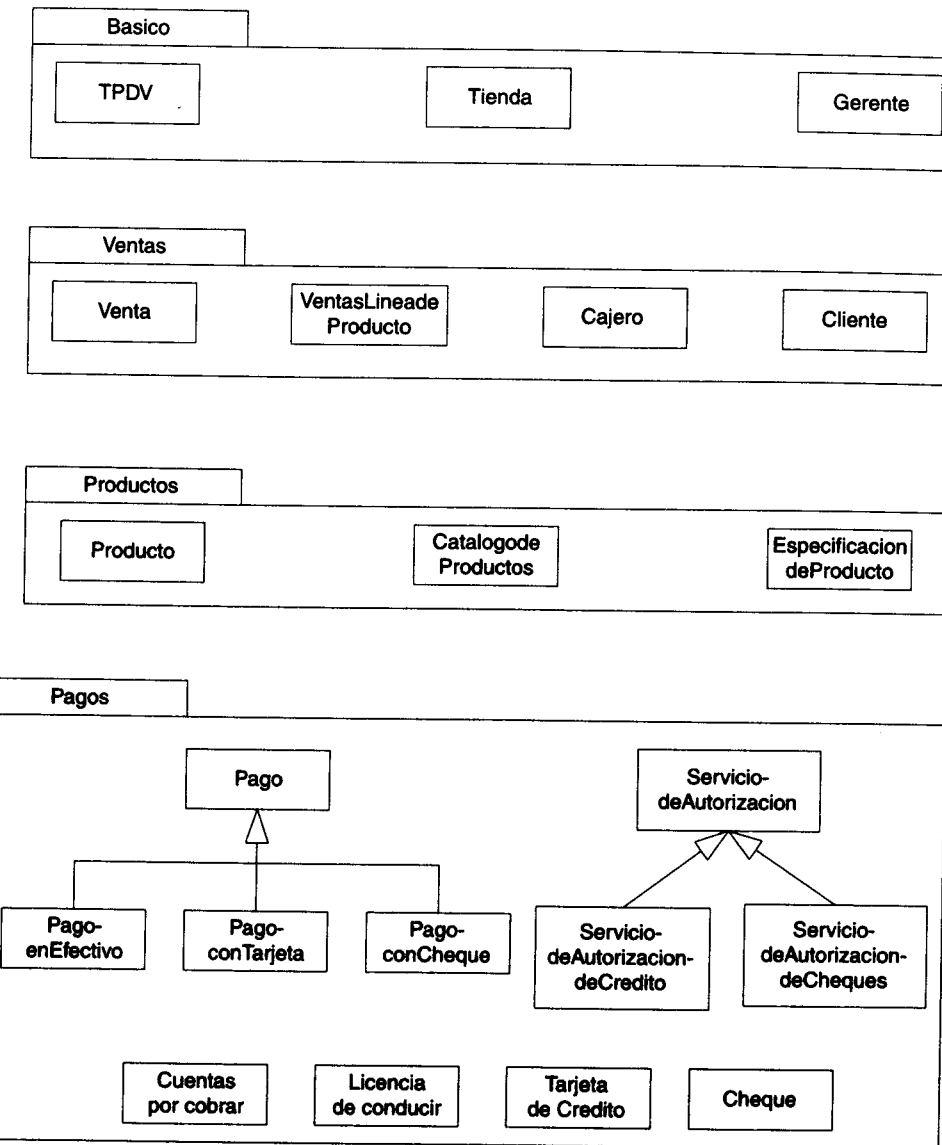


Figura 29.6 Paquetes del dominio.

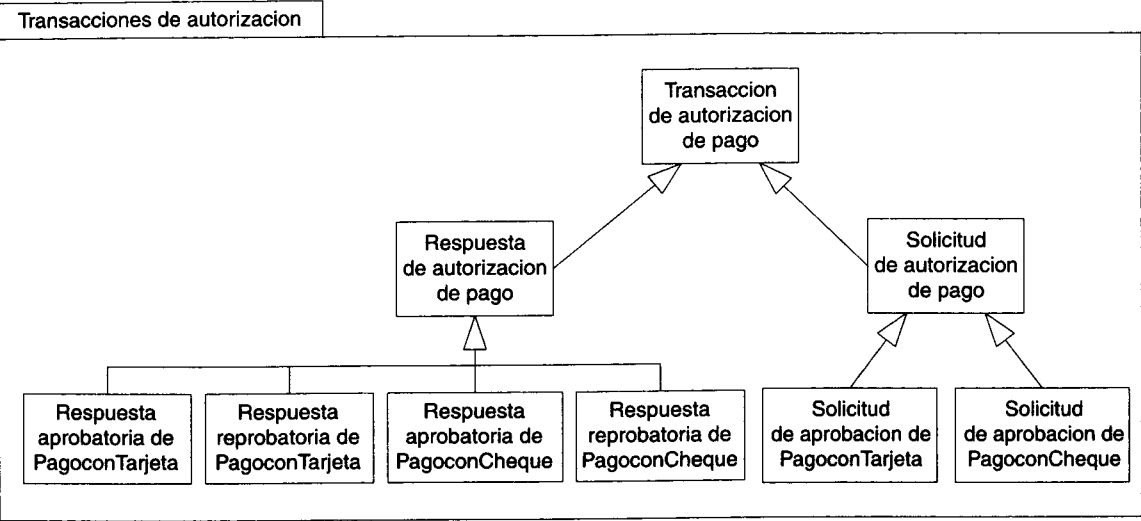


Figura 29.7 Más paquetes del dominio.

REFINAMIENTO DEL MODELO CONCEPTUAL

Objetivos

- Agregar tipos asociativos al modelo conceptual.
- Incorporar las relaciones de agregación.
- Decidir cómo construir los modelos de papeles (funciones que se desempeñan).

30.1 Introducción

En este capítulo vamos a estudiar más ideas y otra notación con que se construyen los modelos conceptuales y vamos a aplicarlos para perfeccionar los aspectos del modelo del punto de venta. En el capítulo siguiente consolidaremos los resultados aquí obtenidos.

30.2 Tipos asociativos

Los siguientes requerimientos del dominio preparan el terreno para los tipos asociativos:

- Los servicios de autorización asignan a las tiendas una identificación comercial que les permite identificarlas durante la comunicación.
- Una solicitud de autorización de pago hecha por una tienda a un servicio de autorización exige incluir la identificación comercial que la identifica ante él.
- Además, una tienda tiene una identificación comercial para cada servicio.

¿En qué parte del modelo conceptual debería residir el atributo de identificación (ID)? Es incorrecto colocar *idComercial* en la *Tienda* porque una *Tienda* puede tener más de un valor de *idComercial*. Lo mismo sucede cuando lo ponemos en *ServiciodeAutorizacion* (figura 30.1).

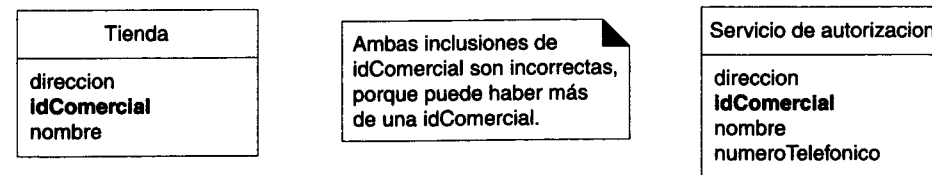


Figura 30.1 Uso incorrecto de un atributo.

De lo que acabamos de decir extraemos el siguiente principio de la construcción de modelos:

En un modelo conceptual, si un tipo T puede tener simultáneamente muchos valores para la misma clase de atributo A, no coloque A en T. Póngalo en otro tipo que esté asociado a T.

Por ejemplo:

- Una *Persona* puede tener muchos números telefónicos. Ponga el número en otro tipo —por ejemplo, *NumeroTelefonico* o *InformaciondeContacto*— y asocie a *Persona* muchos de ellos.

Un modelo mejor, pero todavía no tan útil como podría serlo.

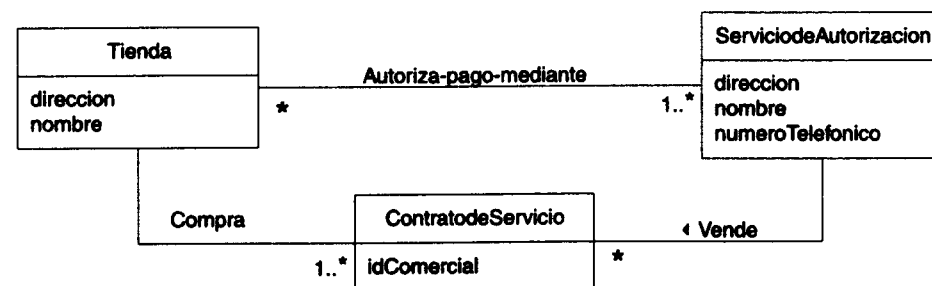


Figura 30.2 Primer intento de construir el modelo del problema de idComercial.

El principio anterior establece que se requiere algo como el modelo de la figura 30.2. En el mundo de los negocios, ¿qué concepto registra formalmente la información referente a los servicios que una compañía ofrece a un cliente?: un *Contrato* o una *Cuenta*.

El hecho de que tanto *Tienda* como *ServiciodeAutorizacion* se relacionen con el *ContratodeServicio* es una indicación dependiente de la relación que existe entre los dos. Podemos considerar la *idComercial* como un atributo relacionado con la asociación entre la *Tienda* y el *ServiciodeAutorizacion*.

Esto desemboca en el concepto de **tipo asociativo** en que podemos agregar características a la asociación.

Podemos modelar el *ContratodeServicio* como un tipo asociativo relacionado con la asociación que existe entre *Tienda* y *ServiciodeAutorizacion*.

En el UML, esto se denota con una línea punteada que de la asociación se dirige al tipo asociativo (figura 30.3).

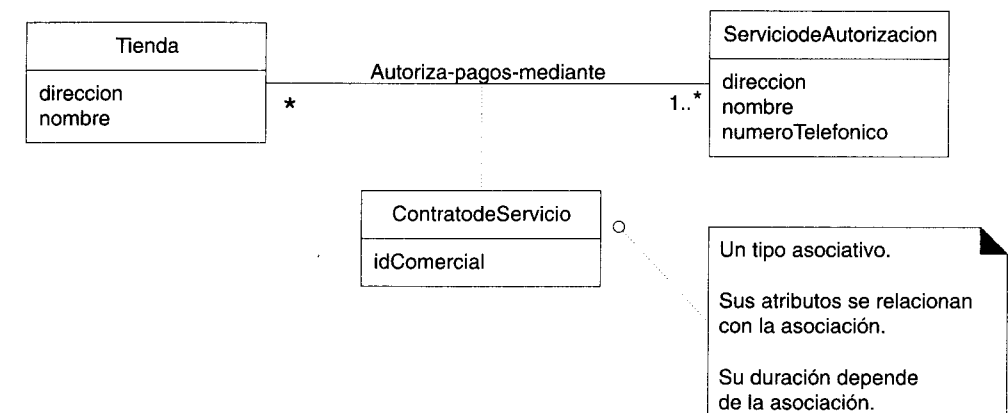


Figura 30.3 Un tipo asociativo.

La figura 30.3 nos transmite visualmente la idea de que un *ContratodeServicio* y sus atributos se relacionan con la asociación existente entre *Tienda* y *ServiciodeAutorizacion*, y de que la duración del *ContratodeServicio* depende de la relación.

30.2.1 Directrices

A continuación ofrecemos algunas normas para agregar los tipos asociativos.

Indicaciones de que un tipo asociativo puede ser útil en un modelo conceptual:

- Un atributo está relacionado con una asociación.
- Las instancias del tipo asociativo presentan una dependencia de toda la vida respecto a la asociación.
- Hay asociaciones de muchos a muchos entre los dos conceptos, y la información se relaciona con la propia asociación.
- Sólo existe una instancia del tipo asociativo entre dos objetos que participan en la asociación.

La presencia de la asociación de muchos a muchos es una indicación común de que un tipo asociativo útil se halla en alguna parte del trasfondo; cuando vea uno, piense en la conveniencia de utilizar un tipo asociativo.

He aquí otros ejemplos de tipos asociativos:

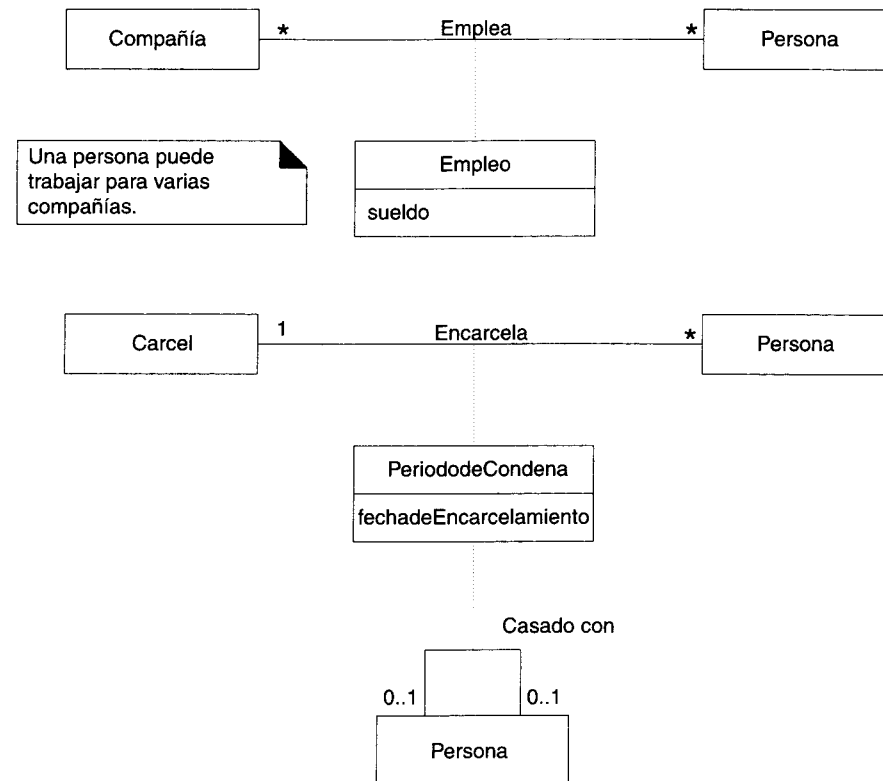


Figura 30.4 Tipos asociativos.

30.3 Agregación y composición

La **agregación** es una clase de asociación con que se modelan las relaciones de parte-todo entre las cosas. Al todo se le llama generalmente **compuesto**; en cambio, las partes no tienen un nombre estándar: suele designárseles simplemente como *parte* o *componente*.

Así, los ensambles físicos se organizan en relaciones de agregación; por ejemplo, una *Mano* agrega o conjunta *Dedos*.

30.3.1 Agregación en el UML

La agregación se muestra en el UML con un símbolo de diamante en blanco o sombreado (en negro) en el extremo correspondiente al compuesto en una asociación de parte-todo (figura 30.5).

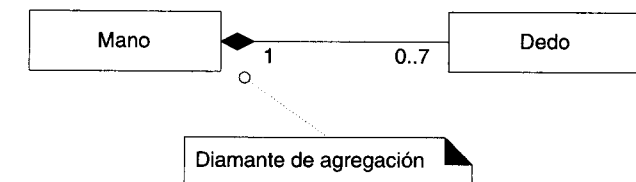


Figura 30.5 Notación de la agregación.

La agregación es una propiedad de un papel de asociación.¹

El nombre de la asociación frecuentemente se excluye en las relaciones de agregación, porque casi siempre se le considera como un *Tiene-parte*. Sin embargo, puede emplearse para proporcionar más detalles semánticos.

30.3.2 Agregación de compuestos: el diamante sombreado

La **agregación compuesta**, o simplemente **composición**, significa que la multiplicidad en el extremo correspondiente al compuesto puede ser al máximo una; la denotamos con un diamante sombreado. Indica que únicamente el compuesto posee la parte y que se encuentra en una jerarquía de partes con estructura de árbol; es la modalidad más frecuente de agregación que encontramos en los modelos.

Por ejemplo, un dedo es parte de no más de una mano y, por lo mismo, el diamante de agregación aparece sombreado para indicar la agregación compuesta (figura 30.6).

¹ Recuerde que cada extremo de una asociación es un papel y que un papel posee varias propiedades: multiplicidad, nombre, navegabilidad y esAgregado.

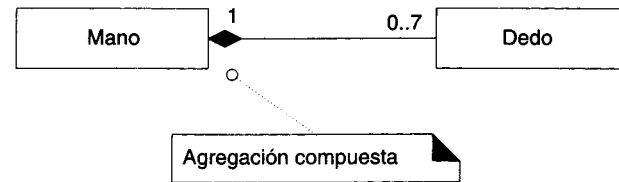


Figura 30.6 Agregación compuesta.

30.3.3 Agregación compartida: diamante en blanco

La **agregación compartida** significa que la multiplicidad en el extremo del compuesto puede ser más de una; se denota con un diamante en blanco. Indica que la parte puede estar en muchas instancias compuestas. La agregación compartida rara vez existe en agregados físicos, pues más bien la encontramos en conceptos no físicos.

Por ejemplo, podemos considerar un paquete de UML para agregar sus elementos. Pero podemos referenciar un elemento en más de un paquete (es propiedad de alguien y se referencia en otros); éste es un ejemplo de la agregación compartida (figura 30.7).

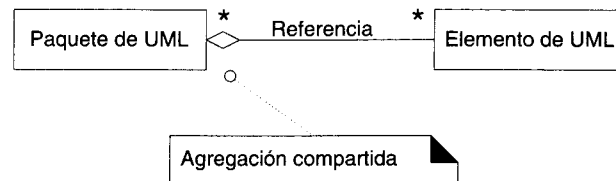


Figura 30.7 Agregación compartida.

30.3.4 Cómo identificar una agregación

En algunos casos la presencia de la agregación es patente: generalmente en los ensamblajes físicos. Pero otras veces no es tan evidente.

Al agregar: en caso de duda, no incluya el elemento.

A continuación se dan algunas directrices que señalan cuándo mostrar la agregación:

Estudie la conveniencia de mostrar la agregación si:

- La duración de la parte es dependiente de la que tiene el compuesto: la parte muestra una dependencia de crear-eliminar respecto al todo.
- Existe un evidente ensamble físico o lógico de parte-todo.
- Algunas propiedades del compuesto se difunden hacia las partes, entre ellas su ubicación.
- Las operaciones aplicadas al compuesto se propagan a las partes: destrucción, movimiento, registro.

Si excluimos el hecho de que algo sea un ensamblado evidente de partes, la siguiente indicación más útil es la dependencia de crear-eliminar que la parte presenta respecto al todo.

30.3.5 Ventaja de mostrar la agregación

No es indispensable ni identificar ni mostrar la agregación; nada impide que excluamos ambas actividades en un modelo conceptual. Recomendamos identificar y mostrar la agregación porque se obtienen los siguientes beneficios, algunos de los cuales se relacionan con la fase de solución del software más que con la de construcción del modelo conceptual. A esto se debe que excluirla en este último no sea muy importante.

- Aclara las restricciones del dominio concernientes a la existencia elegible de la parte independiente del todo. En la agregación compuesta, la parte no puede existir fuera de la duración del todo.
 - Durante la fase de solución, ello repercute en las dependencias crear-eliminar que presentan las clases de software de parte-todo.
- Ayuda a identificar al creador (el compuesto) por medio del patrón Creador de GRASP.
- Las operaciones —como copiar o eliminar— que se aplican al todo deberían propagarse a las partes.
- El identificar un todo en relación con una parte da soporte al encapsulamiento. El patrón No hables con extraños de GRASP sirve para ocultar las partes dentro del todo.

30.3.6 Agregación en el modelo del punto de venta

En el dominio del punto de venta, la instancia *VentasLineadeProducto* puede considerarse parte de un compuesto *Venta*; en una transacción general, la línea de productos

se ve como parte de una transacción agregada (figura 30.8). Además de la conformidad con ese patrón, se observa una dependencia crear-eliminar en la línea de productos de la Venta: su duración no rebasa la de la Venta.

En virtud de una justificación semejante, la instancia *CatalogodeProductos* es un agregado de *EspecificacionesdeProducto*.

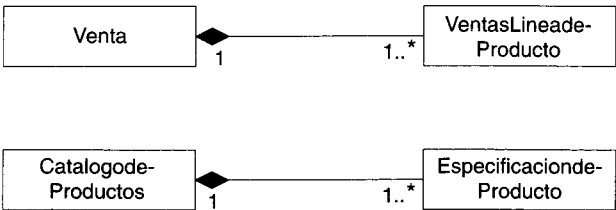


Figura 30.8 Agregación en la aplicación del punto de venta.

Ninguna otra relación es una combinación obligatoria que indique la semántica de parte-todo y una dependencia crear-eliminar; “en caso de duda, no se incluirá”.

30.4 Nombres de los papeles de la asociación

Cada extremo de una asociación es un papel y posee varias propiedades como:

- nombre
- multiplicidad

El nombre del papel identifica un extremo de una asociación y, en teoría, describe la función que los objetos desempeñan en la asociación.

La figura 30.9 contiene ejemplos de nombres de papeles.

No se requiere un nombre explícito de los papeles, aunque resulta de gran utilidad cuando el papel del objeto no es claro. Suele comenzar con una letra minúscula. Si no aparece explícitamente, suponga que el nombre por omisión es igual al nombre del tipo relacionado aunque deberá indicarse con letra minúscula.

Según señalamos ya en este capítulo, los papeles que aparecen en los diagramas de diseño pueden interpretarse como el criterio para asignar nombre a los atributos cuando se genera el código.

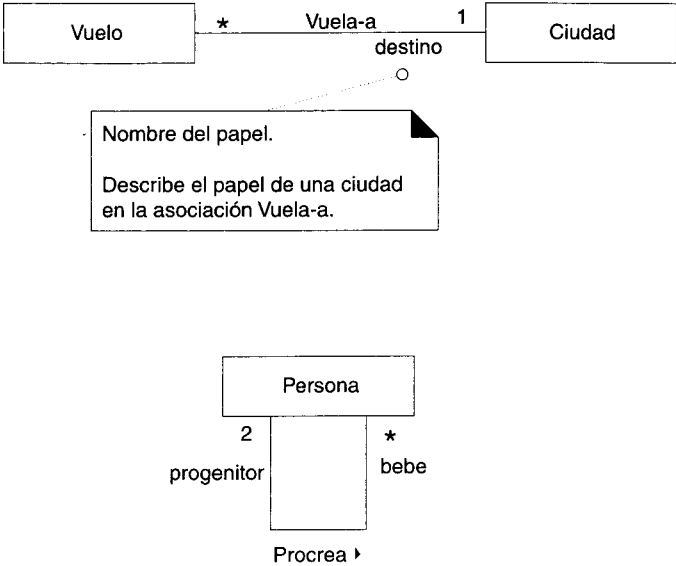


Figura 30.9 Nombres de papeles.

30.5 Los papeles como conceptos y los papeles en las asociaciones

En un modelo conceptual, un papel del mundo real —sobre todo un papel humano— puede modelarse en varias formas, como concepto independiente, o expresarse como un papel en una asociación.¹

Por ejemplo, el papel de un cajero y el de gerente pueden expresarse al menos en las formas que se describen gráficamente en la figura 30.10.

A la primera forma podemos darle el nombre de “papeles en las asociaciones”; al segundo el de “papeles como conceptos”. Ambas modalidades ofrecen ventajas.

Los papeles en las asociaciones son interesantes por constituir un medio bastante preciso de expresar el concepto de que la misma instancia de una persona adopta múltiples papeles (y cambia dinámicamente) en diversas asociaciones. Así, yo —una persona— puedo asumir simultáneamente o en sucesión el papel de maestro, usuario de la tecnología de objetos, padre u otros papeles.

¹ Para no entrar en detalles, prescindimos de otras soluciones excelentes como las que se comentan en [Fowler96].

Por otra parte, los papeles como conceptos ofrecen facilidad y flexibilidad en la incorporación de atributos y asociaciones especiales, así como una semántica complementaria. Más aún, la implementación de papeles como clases aisladas resulta más fácil por las limitaciones de los actuales lenguajes orientados a objetos: es difícil transformar dinámicamente una instancia de una clase en otra o agregar dinámicamente comportamiento y atributos conforme el papel de una persona cambia.¹

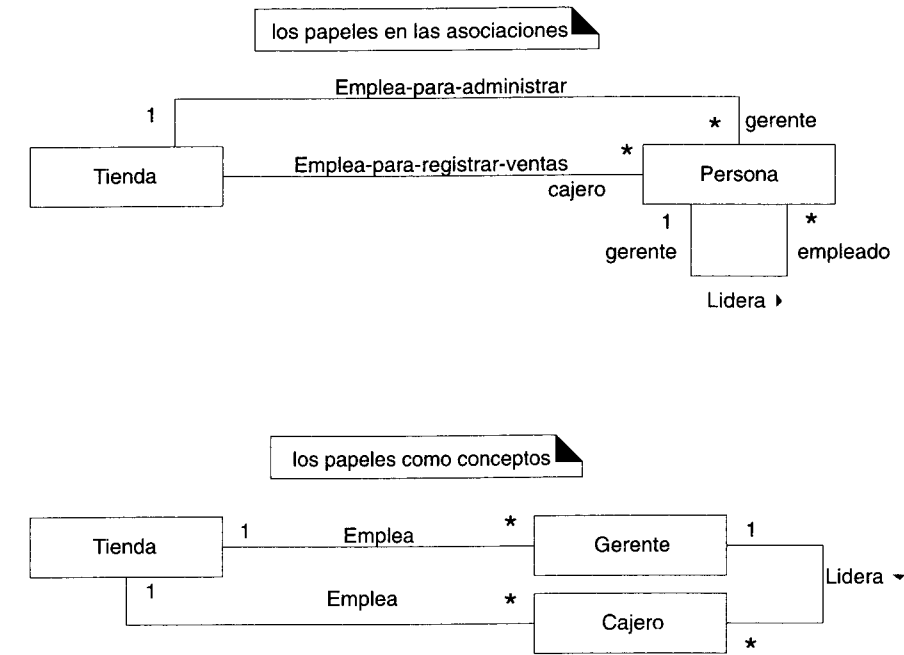


Figura 30.10 Dos formas de modelar los papeles humanos.

Un elemento derivado puede determinarse a partir de otros. Los atributos y las asociaciones son los más comunes de ellos.

Por ejemplo, podemos derivar la información relativa a una *Venta total* y a *VentasLineadeProducto* (figura 30.11). En el UML esto se indica anteponiéndole la diagonal “/” al nombre del elemento.

¹ Lo que se necesita es un lenguaje coloquial que brinde apoyo a los papeles múltiples y de cambio dinámico. Una buena solución parece ser la tecnología de Java basada en agentes.

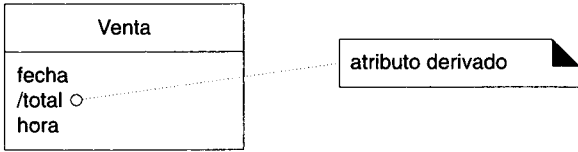


Figura 30.11 Atributo derivado.

¿Cuándo deberíamos mostrar los elementos derivados?

No muestre en un diagrama los elementos derivados, pues aumentan la complejidad sin aportar nueva información. Pero agregue un elemento derivado cuando ocupe un lugar prominente en la terminología del dominio y cuando su exclusión dificulte la comprensión.

Un ejemplo más: una *cantidad de VentaLineadeProducto* es en realidad derivable del número de instancias de *Productos* asociados a la línea (figura 30.12).

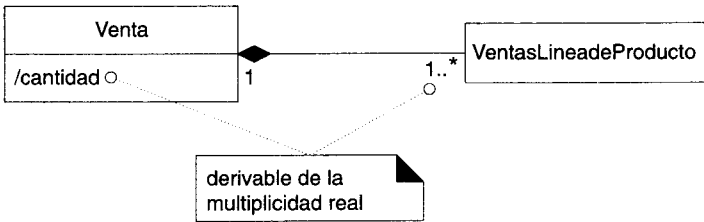


Figura 30.12 Atributo derivado relacionado con la multiplicidad.

30.6 Elementos derivados

30.7 Asociaciones calificadas

Podemos usar un **calificador** en una asociación; su función consiste en distinguir un conjunto de objetos situado en el extremo de la asociación que se basa en el valor del calificador. Se da el nombre de **asociación calificada** a la que posee un calificador.

Por ejemplo, podemos distinguir *EspecificacionesdeProducto* en un *CatalogodeProductos* por su código universal de producto (CUP), como se observa en la figura 30.13 (b). En contraste con la figura 30.13(a) frente a (b), con la calificación se reduce la multiplicidad en el extremo del calificador, generalmente de muchos a uno. La descripción de un calificador en un modelo conceptual señala cómo, en el dominio, los miembros de un tipo se diferencian de otro. En el modelo conceptual no deberían servir para expresar las decisiones de diseño concernientes a las claves de consulta, lo cual sí conviene hacer en los diagramas posteriores de diseño.

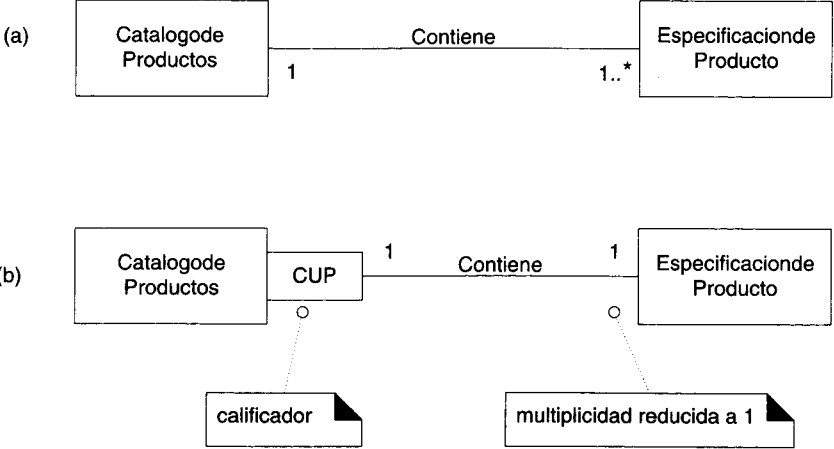


Figura 30.13 Asociación calificada.

Por lo regular los calificadores aportan información nueva y útil, existiendo además el riesgo de caer en la trampa del “pensamiento orientado al diseño”. Pero cuando se utilizan correctamente, pueden aumentar nuestro conocimiento del dominio. Las asociaciones calificadas entre *CatalogodeProductos* y *EspecificaciondeProducto* constituyen un ejemplo razonable de un calificador que añade valor al modelo.

30.8 Asociaciones recursivas o reflexivas

Un concepto puede tener una asociación consigo mismo; a esto se le llama **asociación recursiva** o **reflexiva**.¹

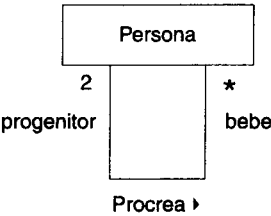


Figura 30.14 Asociación recursiva.

¹ [MO95] limita aún más la definición de las asociaciones reflexivas.

MODELO CONCEPTUAL:
RESUMEN

Objetivos

- Consolidar la información del capítulo anterior referente al modelo conceptual y presentar gráficamente los resultados del modelo del punto de venta.

31.1 Introducción

En este capítulo vamos a reafirmar y explicar visualmente las ideas y la notación de UML que expusimos en los cuatro anteriores. Los resultados se presentan en un modelo conceptual íntegro para el segundo ciclo de desarrollo de la aplicación del punto de venta.

Un modelo conceptual no es intrínsecamente correcto ni erróneo, sino que ofrece una utilidad variable. En consecuencia, contiene decisiones que pueden ponerse en tela de juicio o modificarse por buenas razones, sobre todo al momento de elegir las asociaciones. Pero nos da una buena descripción de los conceptos y de sus relaciones importantes, lo bastante útil para sentar las bases de investigaciones posteriores y del trabajo en la fase de solución. Los estudios subsecuentes seguramente detectarán deficiencias y omisiones en el modelo actual, cosa por lo demás previsible. El modelo se actualizará conforme se vayan obteniendo mejoras.

31.2 Paquete de los conceptos del dominio

En el capítulo 29 se analizaron los motivos del paquete del dominio de alto nivel que se incluye en la figura 31.1. Aquí no se muestran las dependencias del paquete.

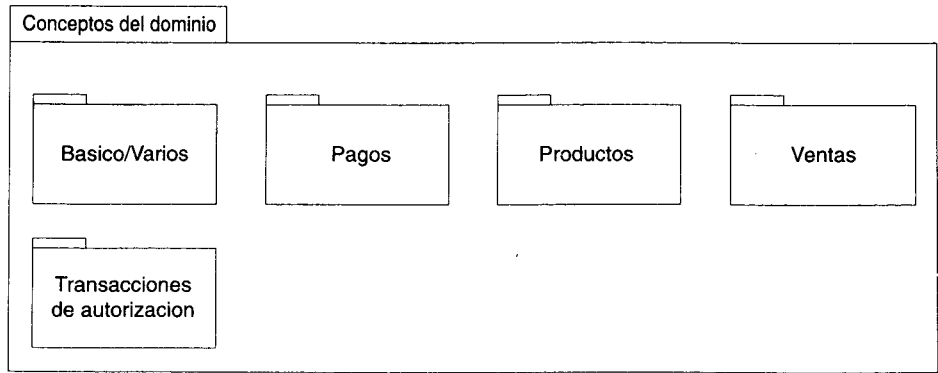


Figura 31.1 Paquete de conceptos del dominio de alto nivel.

31.3 Paquete básico/variantes

Cuando se desee poseer conceptos compartidos por muchos o conceptos que carecen de un sitio bien conocido, conviene servirse de un paquete básico/variantes (figura 31.2). En referencias ulteriores, designaremos a este paquete simplemente como *Básico*.

El paquete no contiene nuevos conceptos ni asociaciones propias de la iteración 2.

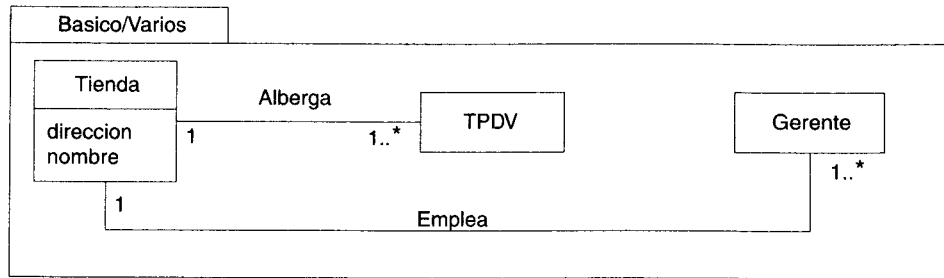


Figura 31.2 Paquete básico.

31.4 Pagos

Igual que en la iteración 1, las nuevas asociaciones se deben principalmente al criterio de la necesidad de conocer. Por ejemplo, es preciso recordar la relación entre un *Pago-conTarjeta* y *TarjetadeCredito*. En cambio, algunas asociaciones se agregan para mejorar la comprensión, como *LicenciadeConducir Identifica Cliente* (figura 31.3).

Nótese que *RespuestadeAutorizaciondePago* se expresa como una clase asociativa. Una respuesta surge de la asociación entre un pago y su servicio de autorización.

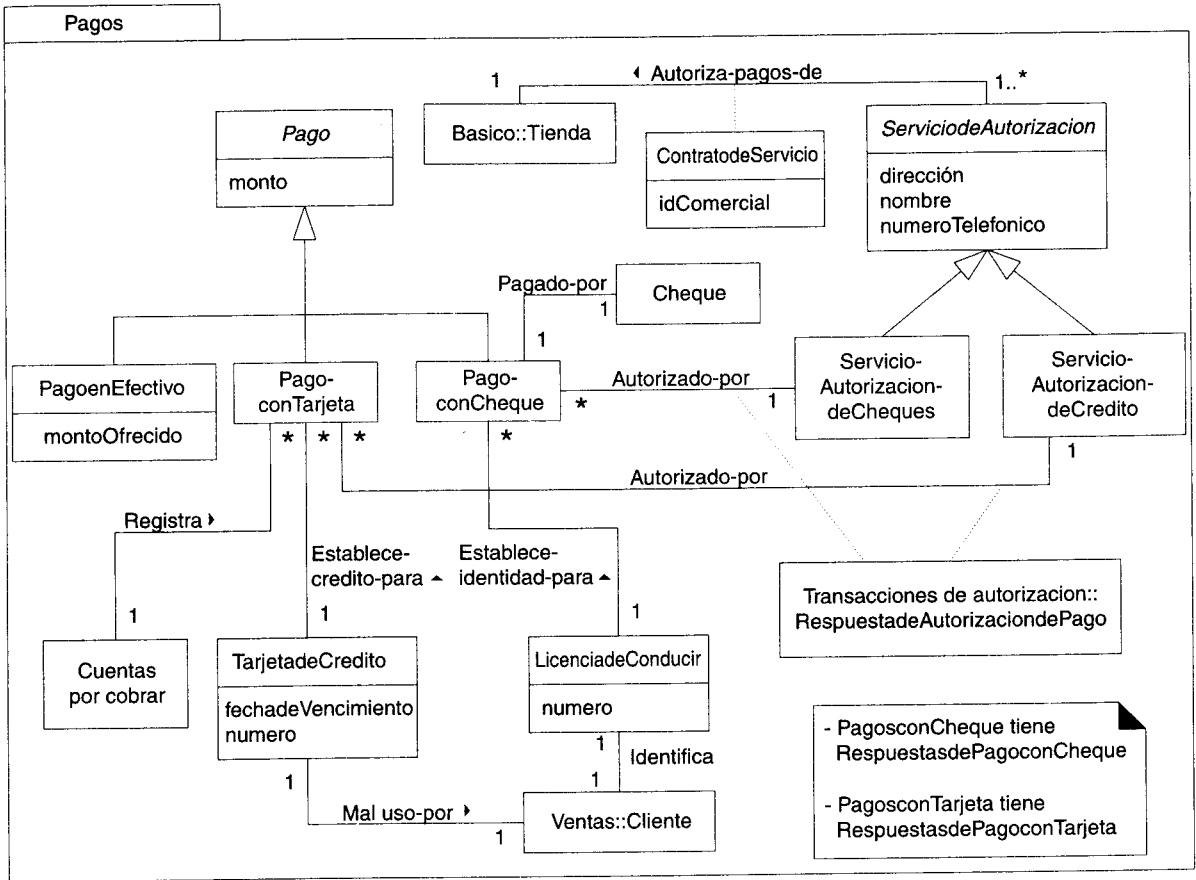


Figura 31.3 Paquete de pagos.

31.5 Productos

A excepción de la agregación compuesta, este paquete no contiene nuevos conceptos ni asociaciones propias de la iteración 2 (figura 31.4).

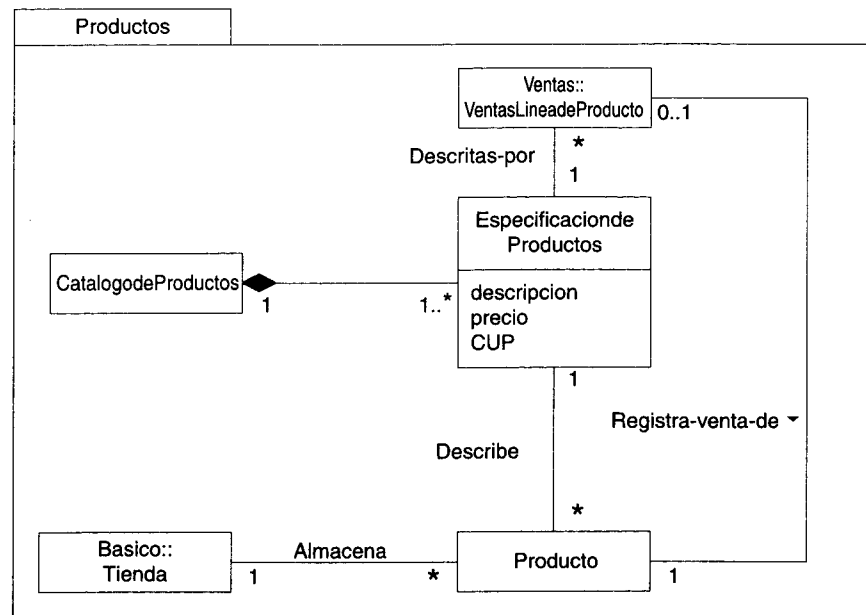


Figura 31.4 Paquete de productos.

31.6 Ventas

Con excepción de la agregación compuesta y los atributos derivados, este paquete no incluye nuevos conceptos ni asociaciones propias de la iteración 2 (figura 31.5).

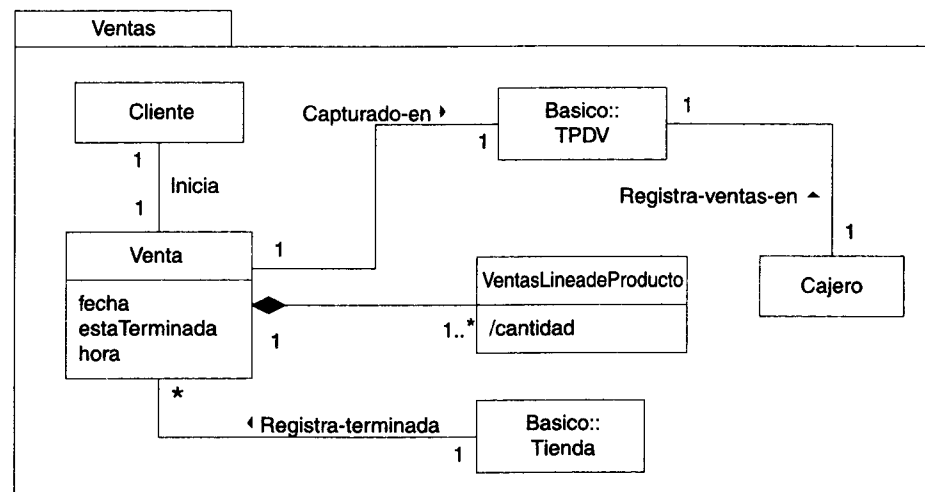


Figura 31.5 Paquete de ventas.

31.7 Transacciones de autorización

Aunque se recomienda asignar nombres significativos a las asociaciones, en algunas circunstancias tal vez ello no sea necesario, especialmente si para la audiencia es evidente el propósito de la asociación. Un caso muy ilustrativo es el constituido por las asociaciones entre los pagos y las transacciones correspondientes. No se especificaron sus nombres, pues supongo que la audiencia al leer el diagrama de tipos de la figura 31.6 entenderá que las transacciones se refieren al pago; si hubiera incorporado sus nombres, simplemente habría atiborrado el diagrama.

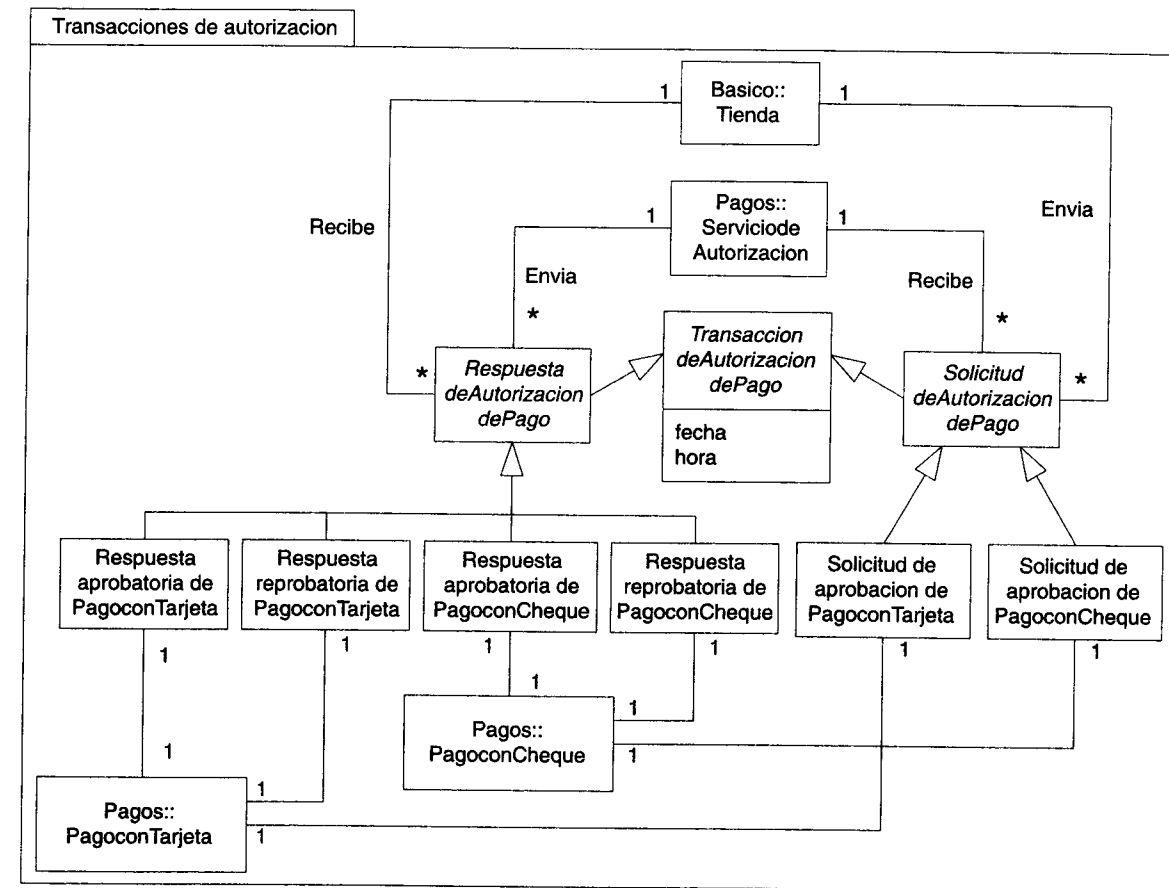


Figura 31.6 Paquete de transacciones de autorización.

¿Resulta demasiado detallado el diagrama anterior porque contiene demasiadas especializaciones? No podemos contestar esta pregunta de manera tajante. El criterio definitivo es la utilidad. Aunque no es incorrecto, ¿de veras contribuye de alguna manera a conocer mejor el dominio? La respuesta a esta pregunta debería influir el número de especializaciones que incorporaremos al modelo conceptual.

COMPORTAMIENTO DE LOS SISTEMAS

Objetivos

- Definir los diagramas de secuencia del sistema y los contratos de su operación en el segundo ciclo de desarrollo.

32.1 Diagramas de secuencia del sistema

En el segundo ciclo de desarrollo, los diagramas de secuencia del sistema (DSS) han de soportar la ramificación de los tipos de pago que se producen en ella. Se agrupan las secuencias comunes en diagramas independientes, como se observa en los siguientes ejemplos.

32.1.1 Inicio común de Comprar Productos

El diagrama de secuencia del sistema de la parte inicial del caso de uso abarca los eventos *introducirProducto* y *terminarVenta*; estos elementos son comunes sin importar el método de pago.

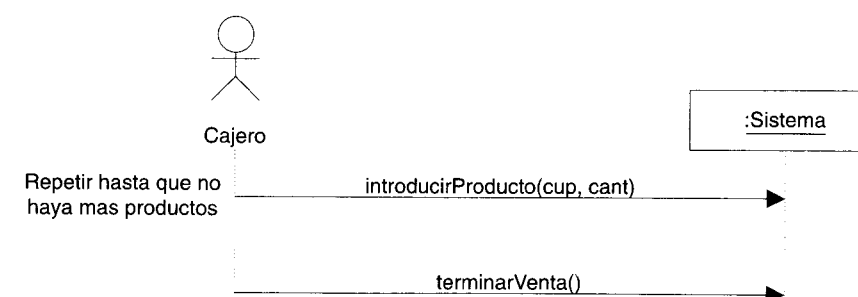


Figura 32.1 Inicio común de los diagramas de secuencia del sistema.

32.1.2 Pago con tarjeta

Este diagrama de secuencia comienza tras un inicio común, cuando se recurre al pago con tarjeta.

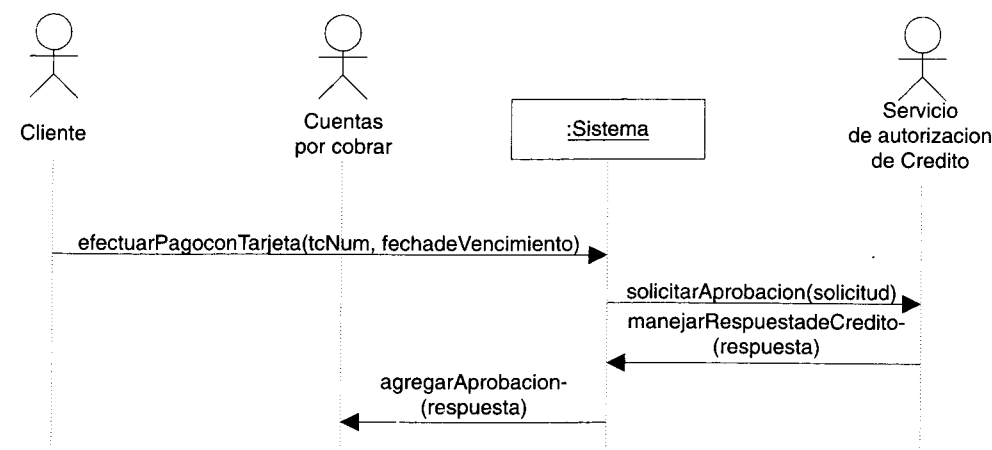


Figura 32.2 Diagramas de secuencia del sistema del pago con tarjeta de crédito.

32.1.3 Pago con cheque

Este diagrama de secuencia empieza tras un inicio común, cuando se recurre al pago con cheque.

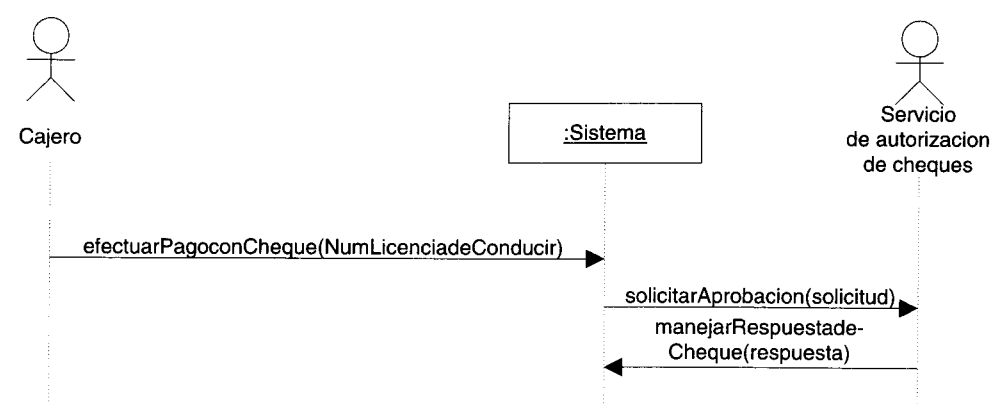


Figura 32.3 Diagramas de secuencia del sistema del pago con cheque.

32.2 Nuevos eventos del sistema

En este ciclo de desarrollo, los nuevos eventos del sistema de la terminal punto de venta son:

- efectuarPagoconTarjeta
- manejarRespuestadeCredito
- efectuarPagoconCheque
- manejarRespuestaconCheque

32.2.1 Cambio de nombre de EfectuarPago

En la primera iteración, el evento y la operación del sistema del pago en efectivo eran *EfectuarPago*. Ahora que los pagos son de distintos tipos, le cambiaremos el nombre y lo llamaremos:

- efectuarPagoenEfectivo

32.3 Contratos

Una operación del sistema se define para cada uno de sus eventos.

Sistema
terminarVenta() introducirProducto() manejarRespuestadeCheque() manejarRespuestadeCredito() efectuarPagoenEfectivo() efectuarPagoconCheque() efectuarPagoconTarjeta()

Hay que escribir contratos para las nuevas operaciones del sistema. Se trata de la mejor conjetura posible respecto a los cambios de estado que ocurrirán en respuesta a un evento del sistema, pero aun así han de considerarse tentativos: durante la fase de diseño probablemente resulten útiles pero incompletos.

En las siguientes secciones describiremos los contratos de operación para la aplicación del punto de venta.

Contrato	
Nombre:	efectuarPagoconTarjeta (tcNum : número fechadeVencimiento : fecha).
Responsabilidades:	Crear y solicitar autorización de un pago con tarjeta de crédito.
Tipo o clase:	Sistema (tipo).
Referencias cruzadas:	
Notas:	Es necesario transformar la solicitud en un registro plano.

Salida: Una solicitud de pago con tarjeta se envía al servicio de autorización de crédito.

Precondiciones: Se terminó la venta actual.

Poscondiciones:

- Se creó un *PagoconTarjeta pgo*.
- Se asoció *pgo* con la *Venta* actual.
- Se creó una *TarjetadeCredito tc*; *tc.numero* = *tcNum*, *tc.fechadeVencimiento* = *fechadeVencimiento*.
- Se asoció *tc* a *pgo*.
- Se creó una *SolicitudPagoconTarjeta spt*.
- Se asoció *pgo* a *spt*.
- Se asoció *spt* a *ServiciodeAutorizaciondeCredito*.

Contrato

Nombre: *manejarRespuestadeCredito*
(*respuesta* : *RespuestaPagoconTarjeta*).

Responsabilidades: Contestar la respuesta del servicio de autorización de crédito. Si la respuesta es aprobatoria, se concluye la venta y se registra el pago en cuentas por cobrar.

Tipo o clase: Sistema (tipo).

Referencias cruzadas:

Notas: *respuesta* es en realidad un registro que ha de ser transformado en una *RespuestaAprobatoriadePagoconTarjeta* o en una *RespuestaReprobatoriadePagoconTarjeta*.

Salida: Si se aprueba la solicitud, la respuesta aprobatoria se envía a cuentas por cobrar.

Precondiciones: La solicitud de pago con tarjeta se envió al servicio de autorización de crédito.

Poscondiciones:

- Si la respuesta fue aprobatoria:
 - Se creó una *aprobación RespuestaAprobatoriadePagoconTarjeta*.
 - Se asoció *aprobación* a *CuentasporCobrar*.
 - Se asoció la *Venta* a la *Tienda* para incorporarla al registro histórico de ventas terminadas.
- En caso contrario, si la respuesta fue reprobatoria:
 - Se creó una *reprobatoria RespuestareprobatoriadePagoconTarjeta*.

Contrato

Nombre: *efectuarPagoconCheque*
(*NumLicenciadeConducir :numero*).

Responsabilidades: Crear y solicitar autorización de pago con cheque.

Tipo o clase: Sistema (tipo).

Referencias cruzadas:

Notas: Es necesario transformar la solicitud en un registro plano.

Salida:

Precondiciones: La venta actual está terminada.

Poscondiciones:

- Se creó un *pgo PagoconCheque*.
- Se asoció *pgo* a la *Venta* actual.
- Se creó una *lc LicenciadeConducir* y *lc.número* = *numLicenciadeConducir*.
- Se asoció *lc* a *pgo*.
- Se creó una *spch SolicituddePagoconCheque*.
- Se asoció *pgo* a *spch*.
- Se asoció *spch* a *ServiciodeAutorizaciondeCheques*.